UNIP – Universidade Paulista

Curso: Ciência da Computação

Polo Alphaville

Sistema PDV integrado com funcionalidades de serviço de entregas delivery

Vitoria Aparecida Almeida DE OLIVEIRA

Santana de Parnaíba

Vitoria Aparecida Almeida DE OLIVEIRA

Sistema PDV integrado com funcionalidades de serviço de entregas delivery

Projeto de Trabalho de Curso apresentado ao Curso de Ciência da Computação

Universidade Paulista

Santana de Parnaíba

2026

RESUMO

Este trabalho tem como objetivo a criação de um sistema de supermercado com integração ao delivery a fim de auxiliar os microempreendedores a adaptarem seu negócio para o sistema de entregas, tendo o poder de customizar suas regras e fluxo de negócio, assim como facilitar a integração do estoque de produto para vendas físicas quanto delivery.

O projeto se apoia com base nos dados dos eventos de isolamento que ocorreram em 2020 durante a época da pandemia da COVID-19 e explora o comportamento e resultados de microempresas e grandes empresas, além de comparar empresas que conseguiram se adaptar à nova realidade e a outras que infelizmente não conseguiram.

Durante o trabalho, é explicado o termo PVD (Ponto de Venda) e suas diferenças dependendo da aplicação. Como objetivo, será criado um sistema de vendas focado em microempresas, com as funções essenciais de caixa, venda, cadastro dos produtos, controle de estoque, cadastro de fornecedores, cadastro de clientes e a opção de delivery, onde o usuário irá customizar os produtos para esse tipo de entrega.

Além disso, o sistema terá níveis de acesso, para administrador e funcionários, podendo separá-los no módulo de venda presencial e delivery. O sistema será desenvolvido em Java, tanto para a interface como para o back-end, onde neste, será utilizado o Java Spring Boot, para facilitar a configuração da API.

Como IDE (Ambiente de Desenvolvimento Integrado), será utilizado o IntelliJ IDEA, bastante comum em projetos com a linguagem Java, apesar de dar suporte para outras linguagens de programação. Para o banco de dados, será utilizado o MySQL, por causa de sua fácil acessibilidade, já que possui uma versão, chamada MySQL Community Edition, que pode ser utilizado gratuitamente.

Para design das telas, será utilizado o Figma, onde será possível visualizar as telas antes da programação, podendo ser aperfeiçoado sem precisar refazer o código.

ABSTRACT

This work aims to create a supermarket system with integration with delivery in order to assist micro-entrepreneurs to adapt their business to the delivery system, having the power to customize its rules and business flow, as well as facilitating the integration of product inventory for physical sales as delivery.

The project is supported by data from isolation events that occurred in 2020 during the time of the COVID-19 pandemic and explores the behavior and outcomes of micro- and large enterprises, besides comparing companies that managed to adapt to the new reality and others that unfortunately did not.

During the work, the term PVD (Point of Sale) and its differences depending on the application is explained. As an objective, a sales system focused on microenterprises will be created, with the essential functions of cash register, sale, registration of products, inventory control, supplier registration, customer registration and the option of delivery, where the user will customize the products for this type of delivery.

In addition, the system will have access levels for administrator and employees, being able to separate them in the module of in-person sales and delivery. The system will be developed in Java, both for the interface and for the back-end, where this will use the Java Spring Boot, to facilitate the configuration of the API.

As an IDE (Integrated Development Environment), IntelliJ IDEA will be used, which is quite common in projects with the Java language, although it supports other programming languages. For the database, MySQL will be used, because of its easy accessibility, since it has a version, called MySQL Community Edition, which can be used for free.

For the design of the screens, will be used Figma, where it will be possible to view the screens before programming and can be improved without having to redo the code.

Key words: System, Technology, Java, POS, SQL, Delivery

SUMÁRIO

INTRODUÇÃO 6
CAPÍTULO 1 - REFERENCIAL TEÓRICO 12
CAPÍTULO 2 – MÉTODO DE PESQUISA 18
CAPÍTULO 3 – DESENVOLVIMENTO 26
Arquitetura da API 28
Mudanças de entidades na API 29
Tecnologias utilizadas na API 29
Criando as Entidades dentro na pasta Domains 29
Criação dos DTOs 35
Criação da Infra 35
Criação dos Repositories 35
Criação do Service 39
Criação dos Controllers 48
CAPÍTULO 4 - RESULTADOS 55
Tela Inicial 55
Tela - Venda PDV 55
Tela - Menu de Consultas 56
Tela - Lista de Produtos 56
Tela - Lista de Fornecedores 57
Tela - Lista de Clientes 57
Tela – Lista de Vendas 57
Tela – Lista de Estoque 58
CAPÍTULO 5 – METODOLOGIA 59
Feedbacks e melhorias 59
Testes e validações 60
Comparação com demais sistemas no mercado 60
REFERÊNCIAS BIBLIOGRÁFICAS 61
CRONOGRAMA 65
ANEXOS 66

INTRODUÇÃO

Durante a pandemia de 2020, a Covid-19, uma doença infecciosa e potencialmente grave, impactou o mundo inteiro. Causada pelo coronavírus SARS-CoV-2, onde por sua vez, possui uma alta taxa de transmissibilidade, teve como resultado diversas medidas protetivas, como a quarentena, proteções com máscaras faciais e isolamento social (CAUNETTO et al., 2022),

Entre essas medidas, tivemos as Fases do Plano de São Paulo, um sistema de cores que foi criado pelo Governo do Estado de São Paulo para atuar conforme o índice de contágio da doença e que serviu para o estado poder adaptar a sua rotina da forma mais segura possível (RADFAHRER, Luli e GURGEL, Dani. Plano São Paulo, 2020)

Diversos outros estados do Brasil também adotaram seus próprios sistemas de cores para alertar a população e decretar as suas medidas, sendo a fase vermelha, sendo considerada entre as maiores fases de risco.

O sistema de cores das Fases do Plano de São Paulo era dividido entre 5 fases, informando, de ordem crescente, o nível de liberação dos comércios e circulação de pessoas. (“Associação Sabesp - Entenda as cores para enfrentarmos a Covid 19”, 2026)

Em 2020, a Sabesp enviou um esclarecimento sobre as fases para os seus colaboradores do estado de São Paulo, onde descreve o plano da seguinte maneira:

Fase 1 (Vermelha): Alerta máximo, liberando apenas os serviços/comércios de atividades essenciais, como mercados, farmácias, escolas, padarias, lotéricas, entre outros.

Fase 2 (Laranja): Liberação parcial de comércios não essenciais, porém atuando com 20% da capacidade e redução de horário para quatro horas seguidas. Nesta fase, shoppings poderiam atuar com exceção da praça de alimentação, onde normalmente se concentra um enorme aglomerado de pessoas.

Fase 3 (Amarela): Fase em que se inicia uma maior flexibilização, aumentando a capacidade dos comércios para 40% e redução de horário para seis horas seguidas. Praças de alimentação, bares e restaurantes poderiam atuar em áreas abertas, para evitar a contaminação.

Fase 4 (Verde): Aumento da capacidade dos comércios para 60% e menos restrições para as atividades econômicas. Continuam estritamente proibidos as atividades que incentivem a aglomeração das pessoas.

Fase 5 (Azul): Fase em que a doença já está devidamente controlada e liberação de todas as atividades.

Percebe-se que os comércios foram extremamente afetados economicamente durante a pandemia, principalmente as micro e pequenas empresas, De acordo com os dados do Instituto Brasileiro de Geografia e Estatística (IBGE), desde a primeira onda da Covid-19 até junho de 2020, 716.372 empresas encerram as suas atividades, sendo 99,8% de menor porte.

Os setores de comércios e serviços foram os mais impactados. Segundo os dados do IBGE, o Instituto de Pesquisa Econômica aplica (IPEA) através de sua estimativa de 240 bilhões do estoque de capital das MPE em dezembro de 2018, calcula-se que o prejuízo com a primeira onda da Covid seja em cerca de R$ 9,1 bilhões e R$ 24,1 bilhões deste estoque.

Em contrapartida, os serviços de Delivery, onde é realizada a entrega domiciliar da compra do cliente, cresceu significativamente durante a pandemia, cerca de 250%, de acordo com dados da consultoria Food Consulting.

De acordo com o portal de notícias do G1, a Uber obteve altos prejuízos com o seu serviço tradicional de carona em 2020, o que condiz com o contexto da pandemia e as medidas protetivas conforme explicado anteriormente. Porém, o seu serviço de Delivery, o Uber Eats, teve um aumento de 224% da sua renda apenas no quarto semestre deste mesmo ano.

Esse aumento de lucro não parou depois da pandemia, em 2021, o Ifood, um dos principais sistemas de Delivery no Brasil, teve um crescimento de 418% desde 2020 e atingiu 5 mil parceiros, entre eles diversos mercados e farmácias, segundo a revista Veja.

De acordo com todos esses dados, podemos concluir que as empresas precisaram se adaptar aos serviços on-line para poderem se manter devidamente lucrativas durante a época da pandemia. E as empresas que não conseguiram ter essa flexibilização, tiveram que fechar as portas.

Sabemos que essa adaptação não é algo simples de ser feito, é necessário treinamento dos funcionários ao sistema de Delivery, onde precisam se adaptar às novas interfaces e fluxo de trabalho, é preciso a contratação de entregadores, onde caso o sistema de delivery seja o do Ifood ou outros sistemas com a dinâmica similar, a empresa pode obter tanto os entregadores terceirizados do delivery, ou utilizar seus próprios entregadores, de acordo com os planos oferecidos pelo Ifood.

Também é preciso investir em uma nova infraestrutura, como um computador adequado para a instalação do sistema e uma boa internet para poder receber os pedidos.

Caso a empresa não tivesse nem ao menos um sistema automatizado de venda e o controle ainda fosse feito da maneira tradicional no papel, a adaptação teria que ser maior ainda, assim como o seu investimento. Isso reflete em como micro e pequenos negócios sofreram durante a pandemia, já que nem todos tiveram a condição de se adaptar aos novos moldes.

A adaptação também exige bastante tempo, principalmente, na fase de implementação, pois é necessário o cadastro na nova plataforma de todos os produtos, readequar os valores de venda, visto que em serviços de delivery, é comum a empresa cobrar uma taxa de comissão em cima do valor total dos pedidos dos parceiros.

No caso do Ifood, a taxa varia de acordo com os seus planos, onde no Plano Básico, a taxa de comissão é de 12% e possui também uma taxa para pagamentos on-line feitos pela plataforma de 3,2%. Neste plano básico, não é possível utilizar os entregadores do Ifood.

Já o Plano Entrega, o parceiro pode contar com os entregadores, mas a taxa de comissão sobe para 23% do valor total do pedido, além de manter a taxa de 3,2% para os pagamentos realizados dentro do delivery.

Tudo isso acaba no aumento do valor final dos produtos, o que pode diminuir o valor competitivo no mercado em relação a empresas que possui seu próprio sistema de delivery e consegue baratear o valor dos seus produtos, já que apesar de ainda terem custos com o serviço, conseguem administrar melhor a sua precificação, e podendo deixar o produto abaixo do valor, comparado com produtos onde a empresa depende de um serviço de entregas a parte.

Essa tese é relativamente fácil de ser comprovada, onde por exemplo, atualmente, no ano de 2026, se você pegar algum comércio que possui seu próprio sistema de delivery e possui o Ifood, muitas das vezes, será mais vantajoso, realizar a compra em seu próprio sistema de entregas.

Ter seu próprio sistema de delivery é geralmente mais vantajoso, principalmente com o passar do tempo, com ele você consegue aumentar a sua margem de lucro e possui mais liberdade para personalizar a experiência do seu cliente a sua maneira. Porém, é necessário investir em mais infraestrutura para poder sustentar o sistema próprio.

De acordo com o UOL Economia, aderir ao seu próprio sistema de delivery é mais indicado para empresas que já possuem sua clientela definida e não tem uma capacidade muito grande de produção, entretanto, já possui uma estrutura mais estável.

Esse perfil de empresa se adequa ao sistema do escopo deste trabalho, implementar um sistema de delivery integrado ao sistema tradicional de mercado de PDV (Ponto de Venda).

O objetivo é criar um sistema PDV onde dentro da plataforma, é possível administrar o serviço de entrega delivery, onde a gestão de estoque será automaticamente monitorada entre os dois tipos de venda, e o usuário poderá configurar sua própria taxa de comissão, podendo ser distribuída entre todos os produtos ou individualmente.

Todas as vendas serão contabilizadas no caixa, com a distinção descritiva de onde foi realizada aquela venda (PDV ou Delivery). Haverá níveis de acesso diferentes dentro do sistema, onde poderá haver um funcionário apenas para acessar a área de delivery e outra apenas para o PDV, ou ambos.

O objetivo desse sistema é oferecer uma maior flexibilidade para os comerciantes poderem utilizar seu próprio sistema de delivery sem precisar criar tudo do zero.

O sistema oferecerá a base para a empresa poder adequar à sua maneira, além de oferecer uma interface familiar tanto para os funcionários do PDV quanto para os que cuidam da venda online, evitando aquele estranhamento com troca de sistema.

Outro ponto importante é que com esse sistema, o gerenciamento de estoque será o mesmo, ou seja, não será necessário realizar dois cadastros de produtos, tanto o PDV quanto o Delivery, utilizarão o mesmo estoque de produtos, podendo diferenciar o valor final deles conforme a taxa de comissão que a empresa poderá inserir no sistema para as vendas on-line.

Em relação a comercialização, o cliente poderá entrar em contato com o comércio através do WhatsApp ou da forma que a empresa achar melhor, visto que esse atendimento será feito através da funcionária do delivery.

Por se tratar de um comércio que não possui grande escala de produção, o e-commerce não se torna uma opção viável, visto que a empresa já possui sua clientela e apenas deseja oferecer uma nova forma de venda aos seus clientes.

O cliente irá realizar o seu pedido solicitando o valor do produto e o funcionário poderá consultá-lo através do sistema na área de Delivery.

Nessa parte, temos duas opções, a funcionária poderá apenas enviar o valor manualmente na conversa com o cliente, ou, caso se trate de um restaurante, a funcionária poderá selecionar a opção Cardápio dentro do sistema, onde os produtos que estarão listados foram categorizados pela própria empresa, e encaminhar o link para o cliente poder consultar os produtos disponíveis.

O fluxo de trabalho é simples, contando com os funcionários para realizar as suas vendas, e o controle final sendo gerenciado pelo sistema. Uma solução prática para as empresas que querem um meio termo entre o modo de venda tradicional e o online.

Inicialmente, haverá uma adaptação inicial como sistemas novos costumam passar, como transmitir todas as informações dos seus produtos e valores ao novo sistema.

Mas passando essa fase, o comércio estará pronto para ter mais flexibilidade das suas vendas sem precisar estar dependente de um sistema de delivery terceirizado ou não consegue se adaptar a tempos difíceis como na época da pandemia.

Com essa nova forma de venda, é provável que haja um aumento no lucro, visto que com isso, os clientes terão mais possibilidades de compras com a empresa.

CAPÍTULO 1 - REFERENCIAL TEÓRICO

Para este referencial teórico, o foco é demonstrar estudos de casos e pesquisas de trabalhos que visam a importância dos sistemas automatizados para os pequenos e micro negócios e como a tecnologia pode impulsionar o crescimento de uma empresa através da otimização de tempo e maior controle e previsibilidade do seu estoque, evitando, assim, prejuízos que poderiam ser evitados.

A importância da utilização da tecnologia em micro e pequenas empresas

Conforme apontado por Ramos e Almeida (2008), um dos principais fatores que impulsionam o crescimento empresarial se dá ao fato da utilização e aplicação da tecnologia, da ciência e da inovação.

O uso e aplicação da tecnologia nas micro e pequenas empresas, é essencial para garantir uma melhoria significativa na gestão de estoque e vendas, evitando prejuízos e garantindo melhorias em relação a custo, qualidade e tempo (ALVES e GILVAN, 2016).

Porém as micro e pequenas empresas possuem mais dificuldade em implementar novas tecnologias nos seus negócios, do que as grandes empresas, devido ao seu baixo fundo de capital de risco (SEBRAE, 2014; 2019).

Em contrapartida, as MPE´s (micro e pequenas empresas), participam expressivamente no Produto Interno Bruto (PIB), sendo um dos principais agentes da economia brasileira, representando 99% dos empreendimentos brasileiros (SEBRAE, 2014; 2019).

Apesar do projeto estar focando em opções de tecnologia via delivery, é importante contextualizar a indústria com a situação atual que as MPE´s veem enfrentando e tentando se adaptar, que é com um termo que surgiu, inicialmente em 2011, na feira de Hannover, na Alemanha, a Indústria 4.0 (SANTOS et al.,2018).

Também conhecido como Quarta Revolução Industrial, fazendo jus as terceiras revoluções anteriores, que consistiam na substituição do trabalho artesanal pelas máquinas à vapor.

A Indústria 4.0 traduz os acontecimentos recentes no uso da tecnologia no ambiente corporativo, como o setor de manufatura mundial, com o uso de Inteligência Artificial, Big Data, Internet das Coisas etc. (Brazilian Journal of Development, Curitiba, v.7, n.5, p. 50614-50637, 2021).

As MPE´s que conseguem implementar novas tecnologias em seus negócios, precisam superar o obstáculo financeiro para o investimento de infraestrutura, implementação do serviço e terceirização de funcionários, além de qualificar seus próprios funcionários para a utilização correta das ferramentas.

Porém quando tudo isso é superado, segundo Albertin et al. (2017), os resultados são bem satisfatórios, como o ganho de produtividade, otimização de tarefas e produções, qualidade e diversificação das formas de obtenção de lucro, podendo comercializar de uma nova perspectiva, facilidade na tomada de decisão, criação de novos modelos de negócios etc.

Mesmo as MPE´s que não possuem condição de um investimento alto inicialmente para a implementação de novas tecnologias, procurando uma alternativa com menor custo para poder usufruir das novidades no mercado, mesmo se for para utilizar modelos mais simples da tecnologia. (ANTUNES, 2019).

Uma tabela que exemplifica bem situação dos MPE´s em relação a Industria 4.0, pode ser vista no Anexo A, em como as empresas que representam quase 100% do empreendimento brasileiro, possuem dificuldades em evoluir seus meios de produção, e que possui carências em diversos aspectos, como o alto custo de implementação comparando com o capital, maiores barreiras de entrada, carência de conhecimento técnico, escassez de políticas de estímulo ao desenvolvimento tecnológico e entre outros:

História da automatização em processos de vendas nos comércios até o surgimento do primeiro sistema PDV

Antes da automatização do processo de vendas e gestão de produtos, era comumente utilizadas as famosas caixas registradoras, que remonta por James Ritty, que em 1879, queria uma forma de prevenir furtos de funcionários e poder

gerenciar de maneira adequada às suas respectivas vendas (“The History of Point of Sale (POS) Systems”, 2025) (ANDRADE, Bruno, 2016).

Conforme o estudo de Bruno Andrade (2016), essas caixas registradoras tinham como função além de contabilizar os trocos e armazenar o dinheiro, elas também registravam o histórico da venda, sinalizando com o memorável sino ao concluí-la.

Aos poucos a evolução da tecnologia iria melhorando a forma como o processo de venda era realizado. Com grande participação National Cash Register (NCR) Company, que incluiu rolos de papel para imprimir o recibo da venda e em 1906, patenteou a ideia de registrar o acúmulo de vendas diariamente, que foi implementada com a ajuda de Charles F. Kettering, através da sua criação da primeira caixa registradora eletrônica. (“The History of Point of Sale (POS) Systems”, 2025)

Diante tantas inovações, aos poucos a humanidade foi construindo o modelo mais comum utilizado hoje em dia nos amplos comércios.

O primeiro sistema PVD (Ponto de Venda) para realizar esse tipo controle foi desenvolvimento e implementado pela IBM no início dos anos 70, como forma de automatizar o processo e proporcionar uma visão mais ampla para os varejistas, podendo acompanhar em tempo real, a situação do estoque e evitar problemas como falta de produto, levando a insatisfação do cliente (“A evolução do PDV! - Blog Linx”, 2019).

O sistema de vendas PDV e seus tipos

O PDV é um sistema que veio evoluindo conforme décadas e atualmente podemos atribuir maior complexidade a ele, comparado com ao século XX. Se antes o objetivo era apenas registrar a venda, hoje em dia, ele é responsável por toda a transação e tipos de pagamentos, pelo gerenciamento de estoque, fechamento do caixa, emissão de nota fiscal, emissão de boletos, notificações de transações via e-mail ao cliente, cadastro de clientes, cadastro de fornecedores e produtos etc.

A principal função ainda é realizar a venda, por isso a sigla PDV, que significa Ponto de Venda. Atualmente, esse termo pode até ser usado em situções além de um caixa no supermercado.

O termo PDV pode ser utilizado para os famosos quiosques nos shoppings e e-commerces (“PDV: quais os tipos e como escolher um software PDV ideal?”, 2026).

Há dois tipos de PDV que podem ser adotados pelas empresas: o PDV Fixo e o PDV Online.

PDV Fixo

É o PDV mais utilizado em comércios mais tradicionais. Consiste em um sistema geralmente offline, que registra a venda apenas no local do comércio, realizando a integração com outros periféricos, como impressoras para imprimir a nota fiscal, leitor de código de barras para ler o código dos produtos, a comunicação com a caixa registradora onde é armazenado o dinheiro físico para abrir a gaveta no momento de dar o troco e a integração com as máquinas de cartões de crédito e débito.

Esse tipo de PDV ainda é muito utilizado, principalmente em supermercados, farmácias, bares e restaurantes.

Além da integração com os periféricos, esse sistema realiza as funções de fechamento do caixa, que registra o saldo final do dia, fazendo um comparativo com o saldo anterior; gerenciamento de estoque, onde assim que uma venda é concluída, realiza a remoção da quantidade vendida daquele produto no estoque, com isso, o usuário tem um controle maior de quantos produtos sobraram e que estão disponíveis a venda, além de poder supervisionar quando um estoque está abaixo do normal e é necessário a reposição.

Alguns sistemas de PDV realizam o controle do estoque e emite um alerta quando o estoque está abaixo do normal, geralmente, permitindo ao usuário, informar a quantidade de estoque mínimo do produto para realizar essa monitoria.

Esse controle otimiza bastante o trabalho do comerciante, pois sem ele, o vendedor teria que contar os produtos manualmente e anotar em um papel todos os dias para conferir quantos produtos foram vendidos e se confere com o estoque atual. Isso é muito suscetível a falha humana e de segurança, pois caso ocorra algum desvio da venda, não será possível comprovar e nem teria como impedi-lo no momento do desvio.

PDV Online

O PDV Online é focado em sistemas para gerenciar as vendas que ocorrem de maneira online, como um e-commerce, por exemplo.

Compartilha das maiorias das funções do PDV Fixo, como registro de venda; gerenciamento dos pagamentos onlines, onde neste caso, monitora o momento que um pagamento é realizado para informar ao vendedor que já está liberado o envio do pedido para o cliente; realiza o controle do estoque dos produtos e o seu cadastro, dando suporte ao vendedor poder gerenciar as suas respectivas quantidades, desativar um produto para não ficar disponível para compra mesmo sem estoque; emissão de nota fiscal e geralmente fornece uma visão geral para o vendedor de quantas vendas foram realizadas em determinado período.

Diferente do PDV Fixo, o Online não tem como objetivo gerenciar os periféricos como impressoras, máquinas de cartões de créditos e outros itens essenciais para uma venda física, visto que o seu foco são as vendas através de sites e aplicativos.

Exemplos de sites que utilizam o PDV Online: MercadoLivre, Shopee, AliExpress, Shein, Magazine Luiza e entre outros. Atualmente, a maioria dos sites que realizam vendas, possuem um sistema de PDV Online para gerenciar a comercialização.

Esse tipo de venda online cresceu bastante nos últimos anos, onde segundo a Abcomm, possui uma média de crescimento anual de 17% no Brasil.

Em 2020, teve um crescimento de 40%, um dado que demonstra o impacto que a pandemia teve no modo de viver da população (“E-commerce no Brasil: dados de um mercado em expansão”, 2026).

Ascensão do Delivery

Apesar do sistema de entrega já existir desde os anos de 1890 com os dabbawala, um grupo de entregadores da Índia que levam marmitas frescas para seus clientes em Mumbai no horário de almoço, inclusive famosos pela sua qualidade nas entregas, o delivery veio se popularizar mesmo durante a época da pandemia (“Quem inventou o Delivery? A história do delivery como você nunca viu”,2023).

Durante a pandemia, o Delivery cresceu cerca de 250%, de acordo com dados da consultoria Food Consulting. Enquanto diversos comércios tiveram um grande impacto negativo nas suas vendas em 2020, o comércio de vendas onlines dispararam.

Um bom exemplo dessa comparação é o caso da Uber, que teve altos prejuízos com o seu serviço tradicional de carona em 2020 e em contrapartida, o seu serviço de Delivery, o Uber Eats, teve um aumento de 224% da sua renda apenas no quarto semestre deste mesmo ano, de acordo com o Portal de notícias do G1.

CAPÍTULO 2 – MÉTODO DE PESQUISA

O método de pesquisa utilizado neste trabalho será a pesquisa descritiva e para a coleta de dados, será aplicada uma abordagem quantitativa, através da análise de materiais, artigos e documentários, podendo ser possível realizar um comparativo de dados de épocas distintas para um melhor entendimento contextual.

O trabalho será baseado em artigos e dados estatísticos apresentados pelo IBGE durante a ascensão do delivery em 2020, além de baseamento em reportagens da época e atuais.

Em relação aos artigos, terá um aprofundamento histórico, como a evolução dos sistemas de vendas PDV, desde a sua origem e causas para o seu surgimento, a fim de compreender sua finalidade e objetivo atualmente.

O principal intuito de utilizar dados quantitativos, como dados estatísticos, é entender o impacto em massa que essa transformação da forma de trabalhar teve aos comerciantes e sua clientela após terem sido pegos de surpresa em 2020 com a pandemia e que não conseguiram se adaptar e utilizarem sistemas de delivery em seus estabelecimentos.

E como mesmo após a opção de vendas on-line, muitas pessoas ainda preferem ir ao mercado presencialmente, sendo necessário um equilíbrio entre as duas formas de vendas.

Outro objetivo da pesquisa é ressaltar como os mais prejudicados economicamente, foram os micros e pequenos empreendedores. Onde, de acordo com Mauro Oddo Nogueira e Rafael de Farias Costa Moreira (2020) em sua publicação utilizando os dados do IBGE, desde a primeira onda da Covid-19 até junho de 2020, 716.372 empresas encerram as suas atividades, sendo 99,8% de menor porte.

Neste escopo de trabalho, os dados quantitativos representam um melhor entendimento geral do impacto dessas transformações nos últimos anos. Visto que dados tão significativos desta maneira, informam como a automatização e a tecnologia é necessária no ramo comercial desde as suas primeiras inovações.

Compreendendo as necessidades e dificuldades dos micros e pequenas empresas, o desenvolvimento do sistema de vendas unindo o PDV (Ponto de Venda e sistema de delivery, será através de tecnologias maduras e bastante utilizadas no mercado, como Java como linguagem de programação e SQL para criação do banco de dados e seu armazenamento, utilizando como IDE (Ambiente de Desenvolvimento Integrado), será utilizado o MySQL Workbench, pela sua opção gratuita com o MySQL Community, que permite utilizar o ecossistema para criar bancos de dados gratuitamente, caso não seja para uso empresarial.

Para a criação da API, será utilizado o Spring Boot, um framework que auxilia no desenvolvimento de aplicações web em Java e microsserviços, otimizando a codificação e facilitando a integração com o banco de dados e testes automatizados, tornando a utilização do Java Spring Framework mais dinâmica, otimizada e simplificada, podendo ser possível a autoconfiguração de forma prática.

Como planejamento do fluxo de atividades e layout das telas, será utilizado o Figma, um software onde é possível montar o design das telas do programa a ser desenvolvido e programar as interações entre elas. Bastante útil para apresentação de ideias.

Ciclo de desenvolvimento de software

Para garantir um sistema de qualidade e evitar retrabalhos durante o projeto, será utilizado a metodologia do ciclo de vida do desenvolvimento de software, também conhecido como SDLC.

Esta metodologia descreve orienta etapas durante o desenvolvimento de um sistema com o intuito de evitar o desperdício de recursos, valores e tempo, além de melhorar a segurança do sistema, garantindo uma separação eficiente de etapas do projeto, que auxilia às equipes a trabalharem de forma mais sinérgica, onde todos possam entender as etapas e realizarem as mesmas práticas.

Entre os benefícios do SDLC para a vida de um sistema de software, temos: custos mais baixos, melhor gerenciamento de riscos, maior eficiência e maior visibilidade para os stakeholder.

A metodologia SDLC divide o projeto em tarefas menores, e essa divisão somado, totalizam 7 etapas, que são amplamente conhecidas no mundo acadêmico e fora dela também. (MICROSOFT, “Visão geral do ciclo de vida do desenvolvimento de software”,2026)

Etapa 1: Planejamento

Nesta etapa de Planejamento, será realizado o levantamento de todos os requisitos necessários para o desenvolvimento do sistema.

Durante a etapa é discutido sobre o problema a ser solucionado, quais dados seriam necessários, a disponibilidade de recursos e tempos, além de alinhar o entendimento com os stakeholder, a fim de evitar maus entendidos durante o projeto e evitar retrabalhos.

Normalmente é realizado uma reunião com as partes interessadas do projeto para realizar esse alinhamento. O objetivo é sair da reunião com todos tendo a compreensão do que o aplicativo irá entregar e solucionar conforme o levantamento.

Etapa 2: Análise

Nesta etapa será realizado a análise dos dados e requisitos levantados na etapa anterior, e pensar em como será realizada a solução do projeto, em como será o fluxo do sistema e alocar os recursos necessários.

Nesta etapa é interessante fazer um diagrama de caso de uso ou fluxograma para visualizar de forma geral, o funcionamento do sistema, podendo ser apresentado ao stakeholder para confirmar as funções do sistema.

Etapa 3: Projeto

Durante a etapa do projeto, será decidido qual arquitetura e componentes serão utilizados no sistema, ou seja, quais as linguagens de programação serão escolhidas, pensando em qual atenderia melhor a solução do caso.

Além disso, são criados os designs das telas, em forma de protótipo do sistema, muito útil para apresentar ao stakeholder, visto que começa a ficar mais próximo do sistema final.

Etapa 4: Desenvolvimento

Como o próprio nome diz, nesta etapa é realizada de fato o desenvolvimento do sistema, escrevendo códigos seguindo toda a lógica discutida nas etapas anteriores, utilizando os diagramas como base e corrigindo erros sistêmicos.

Durante a etapa do desenvolvimento, é importante ter uma boa organização do projeto, utilizando sistemas de controles de versão, como o GitHub, para evitar perdas de arquivos indesejados, atrasando o desenvolvimento.

Etapa 5:Teste

Após o desenvolvimento do projeto ter finalizado a programação de todas as funções requisitadas.

São realizados os testes para verificar se o sistema está funcionando, dentro desta etapa, existem alguns tipos de testes que podem ser utilizados, cada um averiguando um cenário diferente, de forma abrangente ou mais específica:

- Teste unitário: verifica isoladamente se todas as unidades e componentes do código estão funcionando, como por exemplo, métodos criados. - Teste de integração: testa a interação dos componentes entre si.
- Teste de sistema: Alimenta um ambiente de teste para averiguar o comportamento do sistema simulando como se já estivesse em produção.
- Teste do usuário: Seleciona alguns usuários para testar o sistema e fornecer feedbacks de usabilidade e possíveis erros.

Etapa 6: Implantação

Prepara o sistema para o ambiente de produção, criando banco de dados, hospedando em servidores e configurando redes.

Nesta etapa é importante controlar as versões dos sistemas que forem sendo publicados, mantendo um backup do anterior para caso ocorra um erro em uma nova atualização, poder ser desfeita facilmente.

Etapa 7: Manutenção

Esta etapa diferente das anteriores, será sempre continua enquanto o sistema estiver “vivo”, e o que dirá que seu sistema está bem construído, será na maneira como são realizadas as manutenções, se o código estiver bem estruturado, dependendo das manutenções, serão menos trabalhosas e mais eficientes.

O tempo de uma manutenção do sistema é muito importante no custo da empresa, dependendo da relevância e valor agregado do seu sistema.

Para complementar o ciclo de desenvolvimento de software, outras boas práticas são utilizadas durante o desenvolvimento do projeto, mais especificamente no código, como os Design Patterns, que oferecem soluções prévias para futuros possíveis problemas com o código.

Por exemplo, foi comentado anteriormente, na etapa de manutenção, que o tempo para realizar alguma correção ou ajuste no sistema, dita como o sistema está organizado e estruturado, claro que dependendo da solicitação, mudanças no sistema podem realmente ser mais demoradas independente da organização.

Mas problemas comuns e correções frequentes. Devem ser atendidas com facilidade. E os Design Patterns auxiliam bastante nessa questão, como o padrão de projeto Singleton, que instrui que uma classe deve ter apenas uma instância, isso auxilia na manutenção onde seria necessário realizar o ajuste em todos os objetos instanciados daquela classe. (“Design Patterns: o guia definitivo dos Padrões de Projeto”,2021)

Já nesse padrão de projeto, esse problema já foi pensado e solucionado criando apenas uma instância e referenciando ela durante o projeto. Apesar de ser

uma boa prática, deve ser utilizada com cautela, pois nem todo cenário isso será viável e até pode causar o efeito reverso. Por isso, é importante avaliar o contexto do seu sistema antes de implementar algum padrão de projeto.

Sistema de vendas

O sistema de vendas contará com as funcionalidades já tradicionais no mercado, como registro de vendas, cálculo do valor final e troco, cadastro de produtos, controle de estoque, cadastro de fornecedores, relatórios de vendas e fechamento do caixa.

Como adição e inovação ao sistema, além das funções tradicionais de um sistema PDV, será integrado com o sistema de Delivery, onde todos os cadastros de produtos e controle de estoque será integrado com o delivery, evitando o retrabalho de cadastro dos produtos.

Esse módulo de delivery servirá como registro e controle das vendas realizadas desta maneira e não um serviço completo de entrega, como é o caso do Ifood. O serviço de entrega será realizado de forma manual através dos funcionários que utilizarão o sistema.

O objetivo do sistema de Delivery é apenas registrar de forma manual, todas as etapas de preparação do pedido, entrega final e registro da venda, além de gerenciar a taxa de custo inserida pelo próprio usuário, que será automaticamente calculado no valor final do produto que será vendido pelo módulo do Delivery, isso automatiza a precificação dos produtos, pois evitará aquele típico retrabalho de cadastrar novamente os produtos na plataforma do delivery com o valor alterado, visto que o serviço de entregas possuem um custo extra, como entregadores e uma estrutura dedicada.

Esta maneira também facilita a manutenção, pois se o valor do produto for alterado no sistema PDV, ele será automaticamente atualizado no sistema de Delivery com a taxa de custo já calculada.

Tendo essa diferenciação de vendas, será possível gerar relatórios e poder analisar quais vendas foram realizadas de forma presencial, e quais foram pelo

serviço de entrega, podendo gerar insights e auxiliar em tomadas de decisões para o futuro da empresa.

O módulo de vendas pelo Delivery, contará com toda a atualização do pedido, alterada de forma manual pelo usuário, como controle do status daquele pedido. O funcionário que administrará o sistema, poderá enviar um link para o cliente para acompanhamento do pedido.

A conclusão da venda também dependerá da confirmação do funcionário e validação do pagamento.

O intuito desse sistema é oferecer uma opção mais simples para pequenas empresas que desejam aderir a um serviço de entrega, sem precisar ter que se adaptar a um serviço completo de delivery, como Ifood, onde terá que se adequar às suas taxas de comissões e ao sistema em si.

Esse sistema oferece uma porta de entrada ao delivery, mantendo o fluxo de trabalho gerenciado pelos funcionários, mas que ajudará no registro dos pedidos e controle de estoques e geração de relatórios.

O sistema também dará liberdade para a empresa poder inserir sua própria taxa em cima dos produtos vendidos pelo serviço de entrega e terá uma interface familiar tanto para os funcionários que gerenciam apenas o sistema de vendas PDV quanto para os que gerenciam apenas o sistema de Delivery, facilitando a adaptação caso haja uma troca de função, não sendo necessário assimilar uma interface totalmente diferente.

A coleta de dados é um dos passos fundamentais da pesquisa para averiguar as informações, testar hipóteses e concluir comportamentos conforme a análise dos dados.

Para este projeto, a forma como será realizada a coleta de dados será através de uma abordagem quantitativa, analisando materiais da época da pandemia, reportagens, estudos e qualquer outro tipo de fonte verídica.

O intuito deste tipo de coleta é avaliar o comportamento do mercado, principalmente dos pequenos negócios em relação a sua adaptação durante a

pandemia e em como esse acontecimento histórico prejudicou significativamente o micronegócio em relação às grandes empresas.

Não apenas para comprovar o prejuízo desses comerciantes, mas também a ascensão tecnológica através dos deliverys e como empresas e empreendedores que se adaptaram, conseguiram sustentar o seu negócio mesmo em tempos difíceis.

CAPÍTULO 3 – DESENVOLVIMENTO

Esta etapa visa explicar o processo de desenvolvimento do projeto, desde o levantamento de requisitos, tecnologias utilizadas, diagramas para auxiliar no desenvolvimento do banco de dados e explicações para determinadas escolhas, ou seja, o porquê da utilização de determinada tecnologia, **IDEs** e **frameworks**.

Para iniciar o projeto, foquei em filtrar quais são as principais **entidades** que servirão como engrenagem para o sistema. Com bases nos estudos sobre PDV, pude listar as principais **entidades** da seguinte maneira:

- Pedido

- Produto

- Estoque

- Fornecedor

- Funcionário

- Login

Com isso, decidi construir um **diagrama de classes/UML (Unified Modeling Language)** para realizar a modelagem de **objetos** do programa e entender melhor as principais **classes** e como elas irão se comunicar no sistema ao todo.

Para isso, utilizei o Draw.io, um programa gratuito feito para criar diagramas para aplicações, desde **fluxograma, mapa mental, DER e diagrama UML.**

Como a linguagem de programação escolhida foi o **Java**, ter bem definido quais serão as **classes**, seus **atributos** e **métodos** são essenciais para fazer engrenar o sistema.

Neste primeiro momento, foquei em construir o principal do projeto, que é o sistema de vendas, porém já idealizando o módulo de delivery. Por isso, já introduzi na **classe** Vendas, o seu **atributo** Tipo de Venda, para poder realizar a gravação tanto de vendas realizadas fisicamente quando de forma online.

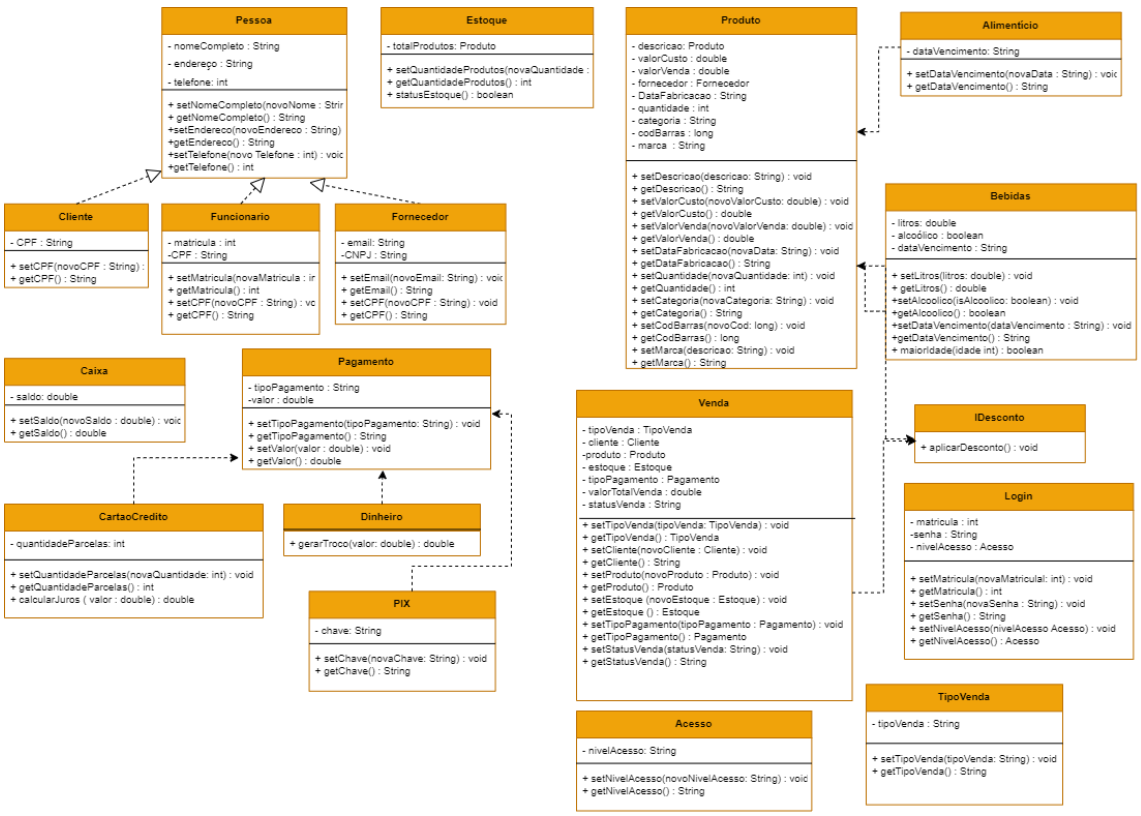
Com este diagrama, pude realizar o processo de **abstração** dos **objetos** que serão utilizados no sistema, ou seja, para as **classes** Cliente, Funcionário e Fornecedor,

irão ser **herdadas** atributos da **classe** Pessoa. Isso auxilia no reaproveitamento de código e otimização.

Essa **abstração** também foi aplicada na **classe** Produto, onde as classes Alimentício e Bebidas derivam das características da classe principal, porém adicionam **atributos** próprios para as duas categorias, como a data de vencimento para produtos alimentícios e informações de litros, teor alcoólico e validação de idade para bebidas.

Por fim, a **abstração** foi utilizada na criação da **classe** Pagamento, com as classes CartaoCredito, Dinheiro e PIX herdando seus principais **atributos**, e acrescentando **métodos** específicos, como o calcularJuros, para a opção de cartão de crédito e gerarTroco, para a opção de dinheiro.

Visando na otimização, também foi criada a **interface** AplicarDesconto, que será um **método** utilizado tanto para a **classe** Produto quanto para a **classe** de Venda. Segue o diagrama completo para mais análise:



Com o UML bem estruturado, o fluxo de como o sistema irá engrenar ficou muito mais otimizada, pois todas as informações já estão descritas. Isso servirá de base para a criação da API.

Arquitetura da API

A arquitetura escolhida para estruturar a API foi a Arquitetura em Camadas, ela consiste em organizar a aplicação separando cada responsabilidade em uma pasta específica, por exemplo:

Controllers: É a camada de apresentação. É nela que ficarão todas as requisições que a aplicação front-end irá chamar. Nela as requisições são validadas, processadas e retornadas com os dados, onde necessário. Ela utilizada as funções criadas na camada **Services** para processar as requisições.

Domains: É a camada onde ficam as entidades que posteriormente serão utilizadas para a criação do banco utilizando o JPA. É nela onde o núcleo do negócio é representado.

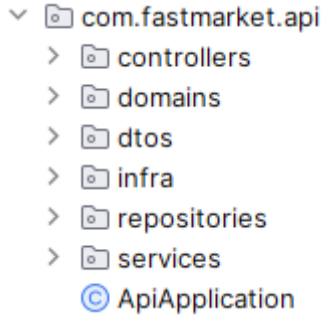
Dtos (Data Transfer Objects): É a camada onde é definido a estrutura dos dados que serão utilizadas na api.

Infra: É a camada onde são tratadas as exceções da api.

Repositories: É a camada que acessa diretamente o banco de dados, através de funções como select, update e delete. Por conta do JPA, algumas funções de select, por exemplo, são otimizadas apenas com a estrutura que é escrito o cabeçalho do método, não sendo necessário escrever a query.

Services: É a camada onde contém a regra do negócio. É nela que terão as funções de criar pedido, adicionar saldo no caixa, cadastrar um produto e etc. Ela faz a ligação com as demais camadas e é onde o **controller** chama para executar alguma função da api para retornar para o cliente (front-end).

Segue o visual da organização das pastas na API:



Mudanças de entidades na API

Conforme a programação, alguns atributos foram alterados e duas novas tabelas foram criadas, a TB_TIPOPEDIDO para especificar o tipo de venda que foi realizada (delivery ou PDV) e a tabela TB_STATUSPEDIDO, para caso a venda tenha sido do tipo Delivery, o cliente poder acompanhar o processo.

Tecnologias utilizadas na API

Para facilitar a programação da API, foi utilizado o **framework Spring Boot**, onde é possível configurar as principais dependências do meu projeto sem precisar escrever todo o código desde o início.

O **Spring Boot** também auxiliou na criação do banco de dados, pois utilizando o **Spring Data JPA**, facilitou a implementação do **JPA** (Java Persistence API), um padrão da linguagem Java que utiliza o mapeamento objeto-relacional (ORM) que serve para mapear a classe e seus atributos como se fossem uma tabela do banco de dados.

Com isso, assim que a api é executada, tendo configurado todo o caminho para o banco de dados, o Sprint Data JPA consegue salvar e atualizar todas as tabelas no banco de dados através de migrações, sendo necessário apenas criar o banco no SQL Server ou **MySQL Workbench** (o que foi utilizado neste projeto).

Outros exemplos de facilidade no desenvolvimento da API utilizando o Spring Boot, é as anotações @Getter, @Setter, @NoArgsConstructor e @AllAargsConstructor. Elas inserem, implicitamente, todos aqueles métodos tradicionais na criação de uma classe. Essas anotações pertencem a dependência Lombok.

Criando as Entidades dentro na pasta Domains

Criação da tabela TB_ACESSO

```
@Entity(name = "Acesso")
@Table(name = "TB_ACESSO")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@EqualsAndHashCode(of = "id")
public class Acesso {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```
    private String acesso;
}
```

Criação da tabela TB_CAIXA

```
@Entity(name = "Caixa")
@Table(name="TB_CAIXA")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@EqualsAndHashCode(of = "id")
public class Caixa {
    @Id
    @GeneratedValue(strategy= GenerationType.IDENTITY)
    private Long id;
    private LocalDateTime dataInicial;
    private double balanco = 0;
    private LocalDateTime dataFinal;
    private boolean isFechado = false;
}
```

Criação da tabela TB_CLIENTE

```
@Entity(name = "Cliente")
@Table(name = "TB_CLIENTE")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@EqualsAndHashCode(of = "id")
public class Cliente {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(unique = true)
    private String cpf;
    private String primeiroNome;
    private String ultimoNome;
    @Column(unique = true)
    private String telefone;
    @Column(unique = true)
    @Email
    private String email;
    private boolean isAtivo = true;
}
```

Criação da tabela TB_ITEMPEDIDO

```
@Entity(name = "ItemPedido")
@Table(name = "TB_ITEMPEDIDO")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@EqualsAndHashCode(of = "id")
public class ItemPedido {
    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private String id;
    @JoinColumn(name = "id_produto")
    @ManyToOne
    private Produto produto;
```

```
    @JoinColumn(name = "id_pedido")
    @ManyToOne
    private Pedido pedido;
    private double valor;
    private int quantidade;
    private double desconto;
}
```

Criação da table TB_TIPOPEDIDO

```
@Entity(name = "TipoPedido")
@Table(name = "TB_TIPOPEDIDO")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@EqualsAndHashCode(of = "id")
public class TipoPedido {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String tipoPedido;
}
```

Criação da tabela TB_LOGIN

```
@Entity(name = "Login")
@Table(name = "TB_LOGIN")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@EqualsAndHashCode(of = "id")
public class Login {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(unique = true)
    private int matricula;
    private String senha;
    @JoinColumn(name = "id_acesso")
    @ManyToOne
    private Acesso acesso;
    private boolean isAtivo = true;
}
```

Criação da tabela TB_METODOPAGAMENTO

```
@Entity(name = "MetodoPagamento")
@Table(name = "TB_METODOPAGAMENTO")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@EqualsAndHashCode(of = "id")
public class MetodoPagamento {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String metodoPagamento;
}
```

Criação da tabela TB_PEDIDO

```
@Entity(name = "Pedido")
@Table(name = "TB_PEDIDO")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@EqualsAndHashCode(of = "id")
public class Pedido {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @JoinColumn(name = "id_cliente")
    @ManyToOne
    private Cliente cliente;
    @JoinColumn(name = "id_metodoPagamento")
    @ManyToOne
    private MetodoPagamento metodoPagamento;
    private LocalDateTime dataVenda;
    @OneToOne
    private StatusPedido statusPedido;
    @ManyToOne
    private TipoPedido tipoPedido;
}
```

Criação da tabela TB_PRODUTO

```
@Entity(name = "Produto")
@Table(name = "TB_PRODUTO")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@EqualsAndHashCode(of = "id")
public class Produto {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String descricao;
    private double precoCusto;
    private double precoVenda;
    @JoinColumn(name = "id_fornecedor")
    @ManyToOne
    private Fornecedor fornecedor;
    private int quantidade;
    @JoinColumn(name = "id_categoriaProduto")
    @ManyToOne
    private CategoriaProduto categoriaProduto;
    @Column(unique = true)
    private Long codigoBarras;
    private String marca;
    private boolean isAtivo = true;
}
```

Criação da tabela TB_CATEGORIAPRODUTO

```
@Entity(name = "CategoriaProduto")
@Table(name = "TB_CATEGORIAPRODUTO")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@EqualsAndHashCode(of = "id")
```

```
public class CategoriaProduto {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String categoriaProduto;
}
```

Criação da tabela TB_ESTOQUE

```
@Entity(name = "Estoque")
@Table(name = "TB_ESTOQUE")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@EqualsAndHashCode(of = "id")
public class Estoque {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @JoinColumn(name = "id_produto")
    @ManyToOne
    private Produto produto;
    private int quantidade;
}
```

Criação da tabela TB_FORNECEDOR

```
@Entity(name = "Fornecedor")
@Table(name = "TB_FORNECEDOR")
@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
@EqualsAndHashCode(of = "id")
public class Fornecedor {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(unique = true)
    private String cnpj;
    @Column(unique = true)
    private String razaoSocial;
    private String nomeFantasia;
    @Column(unique = true)
    private String telefone;
    @Column(unique = true)
    @Email
    private String email;
    private boolean isAtivo = true;
}
```

Criação da tabela TB_STATUSPEDIDO

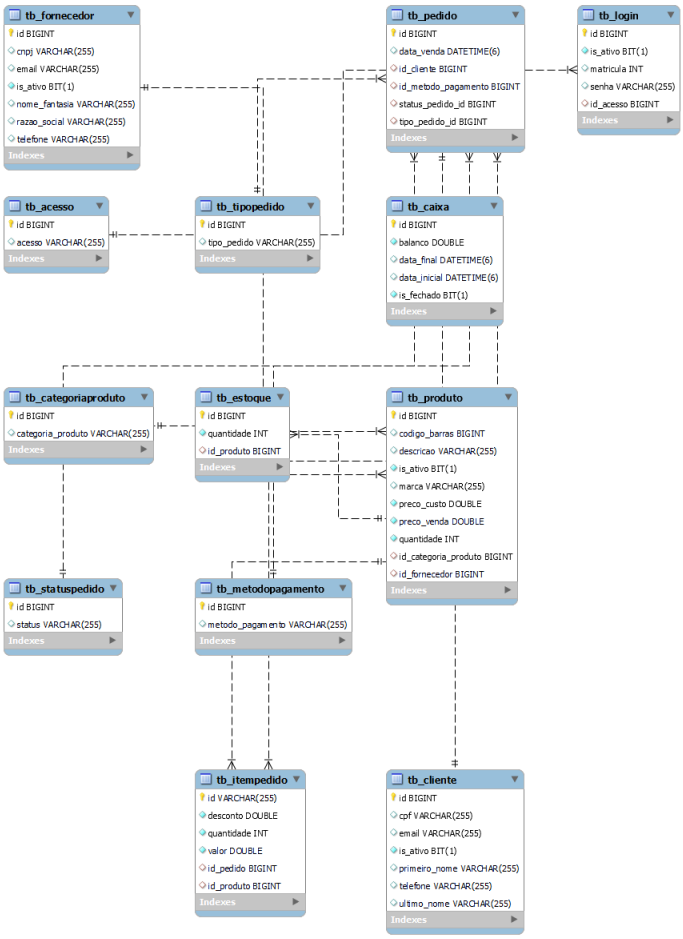
```
@Entity(name = "StatusPedido")
@Table(name = "TB_STATUSPEDIDO")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@EqualsAndHashCode(of = "id")
public class StatusPedido {
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String status;
}
```

Após a criação das classes que servirão para a criação das tabelas no banco de dados utilizando o **JPA**, é preciso configurar no arquivo **application.properties**, o usuário e senha para acessar o banco criado no **MySQL**, indicar o driver a ser utilizado para as migrações, que no caso é o **mysql.cj.jdbc.Driver** e a linguagem do **SQL** a ser analisada, que seria o **MySQLDialect**.

Tendo tudo configurado, basta executar o arquivo que contém a classe **main**, e o serviço da api irá iniciar. Tendo iniciado, através do **application.properties**, a api vai acessar o caminho do banco de dados da máquina e criar todas as tabelas com suas devidas relações, conforme as anotações em JPA que foram inseridas em cada atributo.

Segue o **MER** (Modelo Entidade-Relacionamento), que foi extraído através da engenharia reversa na IDE do **MySQLWorkbench**, após a execução da **API**:



Criação dos DTOs

Os DTOS utilizam a classe Record ao invés do tradicional Class, pois foram pensados justamente para criar esse tipo de classe onde a única finalidade é armazenar os dados. Ou seja, seus atributos são imutáveis após a instância do objeto.

Utilizando os records, também não é necessário adicionar os getters e os setters, pois existe apenas um construtor com todos os atributos. E para puxar o valor de algum atributo, não é utilizado a palavra “get” na frente do nome do atributo.

A fim de não deixar repetitivo, deixo apenas um exemplo de DTO que foi utilizado na API. Todas seguem o mesmo padrão, alterando apenas o nome da classe e os atributos.

Cliente DTO

```
public record ClienteDto(
        @NotNull
        String cpf,
        @NotBlank
        String primeiroNome,
        @NotBlank
        String ultimoNome,
        String telefone,
        String email
){}
```

Criação da Infra

Esta camada serve apenas para tratar as exceções que podem ocorrer na API, por exemplo, o valor pago do pedido for superior ou inferior ao valor calculado conforme o preço de venda dos produtos.

Ou caso o produto do pedido não estiver em estoque. Nestes casos, a api não deve permitir a realização do pedido, e sim, retornar uma mensagem de erro, que será tratado posteriormente na camada Service.

Criação dos Repositories

Conforme tipo anteriormente, os repositories são responsáveis pela manipulação direta no banco de dados. Graças ao JPA, os comandos mais simples, por exemplo, fazer um select através de um ID, não precisam que seja escrito o código propriamente dito. Basta escrever o cabeçalho do método com a nomenclatura

correta. Por exemplo, se eu quiser trazer um produto através do id dele, basta escrever:

```
Optional<Produto> findProdutoById(Long id);
```

AcessoRepository

```
@Transactional
public interface AcessoRepository extends JpaRepository<Acesso, Long> {
    Optional<Acesso> findAcessoById(Long id);
    Optional<Acesso> findAcessoByAcesso(String acessoDescricao);
    @Modifying
    @Query("""
            delete Acesso m
            where m.id = :id
            """)
    void deleteAcesso(Long id);
}
```

CaixaRepository

```
@Transactional
public interface CaixaRepository extends JpaRepository<Caixa, Long> {
    @Modifying
    @Query("""
            update Caixa c
            set c.balanco = :balanco
            where c.id = :id
            """)
    void addBalanco(double balanco,Long id);

    @Modifying
    @Query("""
            update Caixa c
            set c.isFechado = true,
            c.dataFinal = :dataFinal
            where c.id = :id
            """)
    void fecharCaixa(Long id, LocalDateTime dataFinal);

    @Query("""
            select max(balanco) c from Caixa c
            """)
    Object getUltimobalanco();

    @Query("""
            select c from Caixa c
            where c.dataInicial between
            :dateOne and :dateTwo
            order by c.dataFinal
            limit 1
            """)
    Caixa findAtualCaixa(LocalDateTime dateOne, LocalDateTime dateTwo);
    Caixa findAllByisFechadoFalse();
}
```

CategoriaRepository

```
@Transactional
public interface CategoriaProdutoRepository extends
```

```
JpaRepository<CategoriaProduto, Long> {
    Optional<CategoriaProduto> findCategoriaProdutoById(Long id);
    @Modifying
    @Query("""
            delete CategoriaProduto p
            where p.id = :id
            """)
    void deleteCategoriaProduto(Long id);
}
```

ClienteRepository

```
@Transactional
public interface ClienteRepository extends JpaRepository<Cliente, Long> {
    Optional<Cliente> findClienteById(Long id);
    @Modifying
    @Query("""
            update Cliente c
            set c.isAtivo = false
            where c.id = :id
            """)
    void disableCliente(Long id);
}
```

EstoqueRepository

```
@Transactional
public interface EstoqueRepository extends JpaRepository<Estoque, Long> {
    @Modifying
    @Query("""
            update Estoque s
            set s.quantidade = :quantidade
            where s.produto = :produto
            """)
    void updateQuantidade(int quantidade, Produto produto);
}
```

FornecedorRepository

```
@Transactional
public interface FornecedorRepository extends JpaRepository<Fornecedor,
Long> {
    Optional<Fornecedor> findFornecedorById(Long id);
    @Modifying
    @Query("""
            update Fornecedor s
            set s.isAtivo = false
            where s.id = :id
            """)
    void disableFornecedor(Long id);
}
```

ItemPedidoRepository

```
public interface ItemPedidoRepository extends JpaRepository<ItemPedido,
Long> {}
```

MetodoPagamentoRepository

```
@Transactional
public interface MetodoPagamentoRepository extends
JpaRepository<MetodoPagamento, Long> {
    Optional<MetodoPagamento> findMetodoPagamentoById(Long id);
    Optional<MetodoPagamento> findMetodoPagamentoByMetodoPagamento(String
```

```
metodoPagamentoDescricao);
    @Modifying
    @Query("""
            delete MetodoPagamento m
            where m.id = :id
            """)
    void deleteMetodoPagamento(Long id);
}
```

PedidoRepository

```
public interface PedidoRepository extends JpaRepository<Pedido, Long> {
    Optional<Pedido> findPedidoById(Long id);
}
```

ProdutoRepository

```
@Transactional
public interface ProdutoRepository extends JpaRepository<Produto, Long> {
    Optional<Produto> findProdutoById(Long id);
    @Modifying
    @Query("""
            update Produto p
            set p.isAtivo = false
            where p.id = :id
            """)
    void disableProduto(Long id);

    Optional<Produto> findProdutoByCodigoBarras(Long codigoBarras);
}
```

StatusPedidoRepository

```
@Transactional
public interface StatusPedidoRepository extends
JpaRepository<StatusPedido, Long> {
    Optional<StatusPedido> findStatusPedidoById(Long id);
    Optional<StatusPedido> findStatusPedidoByStatus(String status);
    @Modifying
    @Query("""
            delete StatusPedido m
            where m.id = :id
            """)
    void deleteStatusPedido(Long id);
}
```

TipoPedidoRepository

```
@Transactional
public interface TipoPedidoRepository extends JpaRepository<TipoPedido,
Long> {
    Optional<TipoPedido> findTipoPedidoById(Long id);
    Optional<TipoPedido> findTipoPedidoByTipoPedido(String tipoPedido);
    @Modifying
    @Query("""
            delete TipoPedido m
            where m.id = :id
            """)
    void deleteTipoPedido(Long id);
}
```

Criação do Service

Com os Domains, DTOs e Repositories criados, podemos criar a regra de negócio do sistema, ou seja, todas as funções de abrir e fechar o caixa, adicionar saldo, criar um pedido, cadastrar um produto e entre outras.

AcessoService

```
@Service
public class AcessoService {
    @Autowired
    private AcessoRepository acessoRepository;

    public Acesso createAcesso(AcessoDto AcessoDto){
        Acesso newAcesso = new Acesso();
        newAcesso.setAcesso(AcessoDto.acesso());
        this.acessoRepository.save(newAcesso);
        return newAcesso;
    }
    public Acesso updateAcesso(AcessoDto AcessoDto, Long id) throws
Exception {
        Acesso newAcesso = findAcessoById(id);
        newAcesso.setAcesso(AcessoDto.acesso());
        this.acessoRepository.save(newAcesso);
        return newAcesso;
    }
    public Acesso findAcessoById(Long id) throws Exception {
        return this.acessoRepository.findAcessoById(id)
                .orElseThrow(()-> new EntityNotFoundException("Acesso não
encontrado."));
    }

    public Acesso findAcessoByAcessoDescricao(String AcessoDescricao)
throws Exception {
        return this.acessoRepository.findAcessoByAcesso(AcessoDescricao)
                .orElseThrow(()-> new EntityNotFoundException("Acesso não
encontrado."));
    }

    public ResponseEntity<List<Acesso>> getAllAcessos() {
        List<Acesso> AcessoList = this.acessoRepository.findAll();
        return  new ResponseEntity<>(AcessoList, HttpStatus.OK);
    }
    public void deleteAcesso(Long id){
        this.acessoRepository.deleteAcesso(id);
    }
}
```

CaixaService

```
@Service
public class CaixaService {
    @Autowired
    private CaixaRepository caixaRepository;

    public void abrirCaixa(double newBalanco) throws Exception {
        Caixa newCaixa = new Caixa();
        double balanco = newBalanco;
```

```
        if(this.caixaRepository.getUltimobalanco() != null){
            balanco +=
Double.parseDouble(this.caixaRepository.getUltimobalanco().toString());
        }
        newCaixa.setDataInicial(LocalDateTime.now());
        newCaixa.setBalanco(balanco);
        this.caixaRepository.save(newCaixa);
    }

    public void addBalancoCaixa(double balance, LocalDateTime dateSell)
throws Exception {
        LocalDate dateFormmated = dateSell.toLocalDate();
        LocalDateTime initialDate = dateFormmated.atStartOfDay();
        if(this.caixaRepository.findAtualCaixa(initialDate,
dateSell)==null){
            abrirCaixa(balance);
        } else{
            Caixa caixa =
this.caixaRepository.findAtualCaixa(initialDate, dateSell);
            if(caixa.isFechado()){
                abrirCaixa(balance);
            }
            else{
                double newBalance = caixa.getBalanco() + balance;
                this.caixaRepository.addBalanco(newBalance,
caixa.getId());
            }
        }
    }

    public ResponseEntity<List<Caixa>> getAllCaixas(){
        List<Caixa> caixas = this.caixaRepository.findAll();
        return new ResponseEntity<>(caixas, HttpStatus.OK);
    }

    public ResponseEntity<Caixa> closeCaixa(Long id) {
        this.caixaRepository.fecharCaixa(id, LocalDateTime.now());
        return ResponseEntity.noContent().build();
    }
    public ResponseEntity<List<Caixa>> findAllByIsClosedFalse() {
        this.caixaRepository.findAllByisFechadoFalse();
        return ResponseEntity.noContent().build();
    }}
```

CategoriaProdutoService

```
@Service
public class CategoriaProdutoService {
    @Autowired
    private CategoriaProdutoRepository categoriaProdutoRepository;

    public CategoriaProduto createCategoriaProduto(CategoriaProdutoDto
categoriaProdutoDto){
        CategoriaProduto CategoriaProduto = new CategoriaProduto();

CategoriaProduto.setCategoriaProduto(categoriaProdutoDto.categoriaProduto
());
        this.categoriaProdutoRepository.save(CategoriaProduto);
        return CategoriaProduto;
    }
    public CategoriaProduto findCategoriaProdutoById(Long id){
        return
```

```
this.categoriaProdutoRepository.findCategoriaProdutoById(id)
                .orElseThrow(()-> new EntityNotFoundException("Product
category not found."));
    }
    public ResponseEntity<List<CategoriaProduto>>
getAllProductCategories(){
        List<CategoriaProduto> CategoriaProdutoList =
this.categoriaProdutoRepository.findAll();
        return new ResponseEntity<>(CategoriaProdutoList, HttpStatus.OK);
    }
    public CategoriaProduto updateCategoriaProduto(CategoriaProdutoDto
categoriaProdutoDto, Long id){
        CategoriaProduto CategoriaProduto = findCategoriaProdutoById(id);

CategoriaProduto.setCategoriaProduto(categoriaProdutoDto.categoriaProduto
());
        this.categoriaProdutoRepository.save(CategoriaProduto);
        return CategoriaProduto;
    }
    public void deleteCategoriaProduto(Long id){
        this.categoriaProdutoRepository.deleteCategoriaProduto(id);
    }}
```

ClienteService

```
@Service
public class ClienteService {
    @Autowired
    private ClienteRepository clienteRepository;

    public Cliente createCliente(ClienteDto clienteDto) throws Exception{
       Cliente newCliente = new Cliente();
       newCliente.setCpf(clienteDto.cpf());
       newCliente.setEmail(clienteDto.email());
       newCliente.setTelefone(clienteDto.telefone());
       newCliente.setPrimeiroNome(clienteDto.primeiroNome());
       newCliente.setUltimoNome(clienteDto.ultimoNome());

       this.clienteRepository.save(newCliente);
       return newCliente;
    }
    public Cliente updateCliente(ClienteDto clienteDto, Long id) throws
Exception{
        Cliente newCliente =findClienteById(id);
        newCliente.setCpf(clienteDto.cpf());
        newCliente.setEmail(clienteDto.email());
        newCliente.setTelefone(clienteDto.telefone());
        newCliente.setPrimeiroNome(clienteDto.primeiroNome());
        newCliente.setUltimoNome(clienteDto.ultimoNome());

        this.clienteRepository.save(newCliente);
        return newCliente;
    }
    public Cliente findClienteById(Long id) throws Exception{
        return this.clienteRepository.findClienteById(id)
                .orElseThrow(()-> new EntityNotFoundException("Cliente
not found."));

    }

    public ResponseEntity<List<Cliente>> getAllClientes() {
        List<Cliente> clienteList = this.clienteRepository.findAll();
```

```
        return new ResponseEntity<>(clienteList, HttpStatus.OK);
    }
    public void disableCliente(Long id){
        this.clienteRepository.disableCliente(id);
    }}
```

EstoqueService

```
@Service
public class EstoqueService {
    @Autowired
    private EstoqueRepository estoqueRepository;
    @Autowired
    private ProdutoRepository produtoRepository;
    public Estoque createEstoque(EstoqueDto estoqueDto){
        Estoque newEstoque = new Estoque();
        newEstoque.setProduto(estoqueDto.produto());
        newEstoque.setQuantidade(estoqueDto.produto().getQuantidade());

        this.estoqueRepository.save(newEstoque);
        return newEstoque;
    }
    public void updateEstoque(int quantity, Produto produto) throws
Exception {
        produto.setQuantidade(quantity);
        this.estoqueRepository.updateQuantidade(quantity, produto);
    }}
```

FornecedorService

```
@Service
public class FornecedorService {
    @Autowired
    private FornecedorRepository fornecedorRepository;

    public Fornecedor createFornecedor(FornecedorDto fornecedorDto){
        Fornecedor newFornecedor = new Fornecedor();
        newFornecedor.setCnpj(fornecedorDto.cnpj());
        newFornecedor.setEmail(fornecedorDto.email());
        newFornecedor.setTelefone(fornecedorDto.telefone());
        newFornecedor.setRazaoSocial(fornecedorDto.nomeFantasia());
        newFornecedor.setNomeFantasia(fornecedorDto.razaoSocial());

        this.fornecedorRepository.save(newFornecedor);
        return newFornecedor;
    }
    public Fornecedor updateFornecedor(FornecedorDto fornecedorDto, Long
id) throws Exception {
        Fornecedor newFornecedor = findFornecedorById(id);
        newFornecedor.setCnpj(fornecedorDto.cnpj());
        newFornecedor.setEmail(fornecedorDto.email());
        newFornecedor.setTelefone(fornecedorDto.telefone());
        newFornecedor.setRazaoSocial(fornecedorDto.nomeFantasia());
        newFornecedor.setNomeFantasia(fornecedorDto.razaoSocial());

        this.fornecedorRepository.save(newFornecedor);
        return newFornecedor;
    }
    public Fornecedor findFornecedorById(Long id) throws Exception{
        return this.fornecedorRepository.findFornecedorById(id)
                .orElseThrow(()-> new EntityNotFoundException("Fornecedor
not found."));
```

```
    }

    public ResponseEntity<List<Fornecedor>> getAllFornecedors() {
        List<Fornecedor> fornecedorList =
this.fornecedorRepository.findAll();
        return new ResponseEntity<>(fornecedorList, HttpStatus.OK);
    }
    public void disableFornecedor(Long id){
        this.fornecedorRepository.disableFornecedor(id);
    }}
```

ItemPedidoService

```
@Service
public class ItemPedidoService {
    @Autowired
    private ItemPedidoRepository itemPedidoRepository;

    @Autowired
    private CaixaService caixaService;
    @Autowired
    private ProdutoService produtoService;
    @Autowired
    private PedidoService pedidoService;
    @Autowired
    private EstoqueService estoqueService;

    public ItemPedido createItemPedido(ItemPedidoDto itemPedidoDto)
throws Exception {
        Pedido pedido =
pedidoService.findPedidoById(itemPedidoDto.id_pedido());
        Produto produto =
this.produtoService.findProdutoById(itemPedidoDto.id_produto());
        double amountTotal = ((produto.getPrecoVenda() *
itemPedidoDto.quantidade()) - itemPedidoDto.desconto());

        ItemPedido itemPedido = new ItemPedido();

        if (itemPedidoDto.valor() < amountTotal) {
            throw new InsufficientAmountException("Pagamento
insuficiente.");
        }
        if ((itemPedidoDto.valor() - itemPedidoDto.desconto()) !=
amountTotal) {
            throw new InsufficientAmountException("Pagamento superior do
que o necessário.");
        }
        if (itemPedidoDto.quantidade() > produto.getQuantidade()) {
            throw new InsufficientQuantityException("Quantidade
insuficiente no estoque.");
        }
        itemPedido.setPedido(pedido);
        itemPedido.setValor(itemPedidoDto.valor());
        itemPedido.setProduto(produto);
        itemPedido.setQuantidade(itemPedidoDto.quantidade());
        itemPedido.setDesconto(itemPedidoDto.desconto());
        pedido.setDataVenda(LocalDateTime.now());

        this.itemPedidoRepository.save(itemPedido);

        this.caixaService.addBalancoCaixa(amountTotal,
```

```
pedido.getDataVenda());
        this.estoqueService.updateEstoque(produto.getQuantidade() -
itemPedidoDto.quantidade(), produto);
        return itemPedido;
    }

    public ResponseEntity<List<ItemPedido>> getAllItemPedidos() {
        List<ItemPedido> orderList = this.itemPedidoRepository.findAll();
        return new ResponseEntity<>(orderList, HttpStatus.OK);
    }}
```

MetodoPagamentoService

```
@Service
public class MetodoPagamentoService {
    @Autowired
    private MetodoPagamentoRepository metodoPagamentoRepository;

    public MetodoPagamento createMetodoPagamento(MetodoPagamentoDto
metodoPagamentoDto){
        MetodoPagamento newMetodoPagamento = new MetodoPagamento();

newMetodoPagamento.setMetodoPagamento(metodoPagamentoDto.metodoPagamento(
));
        this.metodoPagamentoRepository.save(newMetodoPagamento);
        return newMetodoPagamento;
    }
    public MetodoPagamento updateMetodoPagamento(MetodoPagamentoDto
metodoPagamentoDto, Long id) throws Exception {
        MetodoPagamento newMetodoPagamento = findMetodoPagamentoById(id);

newMetodoPagamento.setMetodoPagamento(metodoPagamentoDto.metodoPagamento(
));
        this.metodoPagamentoRepository.save(newMetodoPagamento);
        return newMetodoPagamento;
    }
    public MetodoPagamento findMetodoPagamentoById(Long id) throws
Exception {
        return this.metodoPagamentoRepository.findMetodoPagamentoById(id)
                .orElseThrow(()-> new EntityNotFoundException("Method
Payment not found."));
    }

    public MetodoPagamento
findMetodoPagamentoByDescriptionMetodoPagamento(String
descriptionMetodoPagamento) throws Exception {
        return
this.metodoPagamentoRepository.findMetodoPagamentoByMetodoPagamento(descr
iptionMetodoPagamento)
                .orElseThrow(()-> new EntityNotFoundException("Method
Payment not found."));
    }

    public ResponseEntity<List<MetodoPagamento>> getAllMetodoPagamentos()
{
        List<MetodoPagamento> metodoPagamentoList =
this.metodoPagamentoRepository.findAll();
        return  new ResponseEntity<>(metodoPagamentoList, HttpStatus.OK);
    }
    public void deleteMetodoPagamento(Long id){
        this.metodoPagamentoRepository.deleteMetodoPagamento(id);
    }}
```

PedidoService

```
@Service
public class PedidoService {
    @Autowired
    private PedidoRepository pedidoRepository;

    @Autowired
    private CaixaService caixaService;
    @Autowired
    private ProdutoService produtoService;
    @Autowired
    private ClienteService clienteService;
    @Autowired
    private MetodoPagamentoService metodoPagamentoService;
    @Autowired
    private EstoqueService estoqueService;

    public Pedido createPedido(PedidoDto pedidoDto) throws Exception {
       Pedido pedido = new Pedido();
       if(pedidoDto.id_cliente() !=null){
           Cliente cliente =
this.clienteService.findClienteById(pedidoDto.id_cliente());
           pedido.setCliente(cliente);
       }
       MetodoPagamento metodoPagamento =
this.metodoPagamentoService.findMetodoPagamentoById(pedidoDto.id_metodoPa
gamento());
       pedido.setMetodoPagamento(metodoPagamento);
       pedido.setDataVenda(LocalDateTime.now());
       this.pedidoRepository.save(pedido);
       return pedido;
    }
    public Pedido findPedidoById(Long id){
        return this.pedidoRepository.findPedidoById(id)
                .orElseThrow(()-> new EntityNotFoundException("Pedido not
found."));
    }
    public ResponseEntity<List<Pedido>> getAllPedidos() {
        List<Pedido> pedidoList = this.pedidoRepository.findAll();
        return new ResponseEntity<>(pedidoList, HttpStatus.OK);
    }}
```

ProdutoService

```
@Service
public class ProdutoService {
    @Autowired
    private FornecedorService fornecedorService;
    @Autowired
    private EstoqueService estoqueService;
    @Autowired
    private CategoriaProdutoService categoriaProdutoService;
    @Autowired
    private ProdutoRepository produtoRepository;

    public Produto createProduto(ProdutoDto produtoDto) throws Exception{
        Fornecedor fornecedor =
fornecedorService.findFornecedorById(produtoDto.id_fornecedor());
        CategoriaProduto ProdutoCategory =
categoriaProdutoService.findCategoriaProdutoById(produtoDto.id_categoriaP
```

```
roduto());
        Produto newProduto = new Produto();
        newProduto.setCategoriaProduto(ProdutoCategory);
        newProduto.setMarca(produtoDto.marca());
        newProduto.setDescricao(produtoDto.descricao());
        newProduto.setCodigoBarras(produtoDto.codigoBarras());
        newProduto.setPrecoCusto(produtoDto.precoCusto());
        newProduto.setPrecoVenda(produtoDto.precoVenda());
        newProduto.setQuantidade(produtoDto.quantidade());
        newProduto.setFornecedor(fornecedor);

        this.produtoRepository.save(newProduto);

        EstoqueDto newEstoqueDto = new EstoqueDto(newProduto,
newProduto.getQuantidade());
        this.estoqueService.createEstoque(newEstoqueDto);
        return newProduto;
    }
    public Produto updateProduto(ProdutoDto produtoDto, Long id) throws
Exception{
        Fornecedor fornecedor =
fornecedorService.findFornecedorById(produtoDto.id_fornecedor());
        CategoriaProduto ProdutoCategory =
categoriaProdutoService.findCategoriaProdutoById(produtoDto.id_categoriaP
roduto());
        Produto newProduto = findProdutoById(id);
        newProduto.setCategoriaProduto(ProdutoCategory);
        newProduto.setMarca(produtoDto.marca());
        newProduto.setDescricao(produtoDto.descricao());
        newProduto.setCodigoBarras(produtoDto.codigoBarras());
        newProduto.setPrecoCusto(produtoDto.precoCusto());
        newProduto.setPrecoVenda(produtoDto.precoVenda());
        newProduto.setQuantidade(produtoDto.quantidade());
        newProduto.setFornecedor(fornecedor);

        this.produtoRepository.save(newProduto);

        EstoqueDto newEstoqueDto = new EstoqueDto(newProduto,
newProduto.getQuantidade());
        this.estoqueService.createEstoque(newEstoqueDto);
        return newProduto;
    }
    public Produto findProdutoById(Long id) throws Exception{
        return this.produtoRepository.findProdutoById(id)
                .orElseThrow(()-> new EntityNotFoundException("Produto
not found."));

    }
    public Produto findProdutoByIBarCode(Long barcode) throws Exception{
        return this.produtoRepository.findProdutoByCodigoBarras(barcode)
                .orElseThrow(()-> new EntityNotFoundException("Produto
not found."));

    }
    public ResponseEntity<List<Produto>> getAllProdutos() {
        List<Produto> produtoList = this.produtoRepository.findAll();
        return new ResponseEntity<>(produtoList, HttpStatus.OK);
    }
    public void disableProduto(Long id){
        this.produtoRepository.disableProduto(id);
    }}
```

StatusPedidoService

```
@Service
public class StatusPedidoService {
    @Autowired
    private StatusPedidoRepository statusPedidoRepository;

    public StatusPedido createStatusPedido(StatusPedidoDto
statusPedidoDto){
        StatusPedido newStatusPedido = new StatusPedido();
        newStatusPedido.setStatus(statusPedidoDto.status());
        this.statusPedidoRepository.save(newStatusPedido);
        return newStatusPedido;
    }
    public StatusPedido updateStatusPedido(StatusPedidoDto
statusPedidoDto, Long id) throws Exception {
        StatusPedido newStatusPedido = findStatusPedidoById(id);
        newStatusPedido.setStatus(statusPedidoDto.status());
        this.statusPedidoRepository.save(newStatusPedido);
        return newStatusPedido;
    }
    public StatusPedido findStatusPedidoById(Long id) throws Exception {
        return this.statusPedidoRepository.findStatusPedidoById(id)
                .orElseThrow(()-> new EntityNotFoundException("Order
Status not found."));
    }

    public StatusPedido findStatusPedidoByStatusPedidoDescription(String
status) throws Exception {
        return
this.statusPedidoRepository.findStatusPedidoByStatus(status)
                .orElseThrow(()-> new EntityNotFoundException("Order
Status not found."));
    }

    public ResponseEntity<List<StatusPedido>> getAllStatusPedido() {
        List<StatusPedido> statusPedidoList =
this.statusPedidoRepository.findAll();
        return  new ResponseEntity<>(statusPedidoList, HttpStatus.OK);
    }
    public void deleteStatusPedido(Long id){
        this.statusPedidoRepository.deleteStatusPedido(id);
    }}
```

TipoPedidoService

```
@Service
public class TipoPedidoService {
    @Autowired
    private TipoPedidoRepository tipoPedidoRepository;

    public TipoPedido createTipoPedido(TipoPedidoDto tipoPedidoDto){
        TipoPedido newTipoPedido = new TipoPedido();
        newTipoPedido.setTipoPedido(tipoPedidoDto.tipoPedido());
        this.tipoPedidoRepository.save(newTipoPedido);
        return newTipoPedido;
    }
    public TipoPedido updateTipoPedido(TipoPedidoDto tipoPedidoDto, Long
id) throws Exception {
        TipoPedido newTipoPedido = findTipoPedidoById(id);
        newTipoPedido.setTipoPedido(tipoPedidoDto.tipoPedido());
```

```
        this.tipoPedidoRepository.save(newTipoPedido);
        return newTipoPedido;
    }
    public TipoPedido findTipoPedidoById(Long id) throws Exception {
        return this.tipoPedidoRepository.findTipoPedidoById(id)
                .orElseThrow(()-> new EntityNotFoundException("Order Type
not found."));
    }

    public TipoPedido findTipoPedidoByTipoPedidoDescription(String
tipoPedido) throws Exception {
        return
this.tipoPedidoRepository.findTipoPedidoByTipoPedido(tipoPedido)
                .orElseThrow(()-> new EntityNotFoundException("Order Type
not found."));
    }

    public ResponseEntity<List<TipoPedido>> getAllTipoPedidos() {
        List<TipoPedido> tipoPedidoList =
this.tipoPedidoRepository.findAll();
        return  new ResponseEntity<>(tipoPedidoList, HttpStatus.OK);
    }
    public void deleteTipoPedido(Long id){
        this.tipoPedidoRepository.deleteTipoPedido(id);
    }}
```

Criação dos Controllers

Por fim, depois de todas as funcionalidades criadas, podemos criar de fato as requisições que seram chamadas pelo cliente/front-end. É nessa camada que serão definidos os end-points que devem ser especificados na requisição. Nela teremos todo o CRUD (Create, Read, Update and Delete) da aplicação.

AcessoController

```
@RestController
@RequestMapping("/acesso")
public class AcessoController {
    @Autowired
    private AcessoService AcessoService;
    @PostMapping
    public ResponseEntity<Acesso> createAcesso(@RequestBody @Valid
AcessoDto AcessoDto){
        Acesso newAcesso = this.AcessoService.createAcesso(AcessoDto);
        return new ResponseEntity<>(newAcesso, HttpStatus.CREATED);
    }
    @GetMapping
    public ResponseEntity<List<Acesso>> getAllAcessos(){
        return this.AcessoService.getAllAcessos();
    }
    @GetMapping("/{id}")
    public ResponseEntity<Acesso> getAcessoById(@PathVariable Long id)
throws Exception {
        Acesso Acesso = AcessoService.findAcessoById(id);
        return new ResponseEntity<>(Acesso, HttpStatus.OK);
    }
    @GetMapping("/AcessoDescription/{AcessoDescription}")
```

```
    public ResponseEntity<Acesso>
getAcessoByAcessoDescription(@PathVariable String AcessoDescription)
throws Exception {
        Acesso Acesso =
AcessoService.findAcessoByAcessoDescricao(AcessoDescription);
        return new ResponseEntity<>(Acesso, HttpStatus.OK);
    }
    @PutMapping("/{id}")
    public ResponseEntity<Acesso> updateAcesso(@RequestBody @Valid
AcessoDto AcessoDto, @PathVariable Long id) throws Exception {
        Acesso newAcesso = this.AcessoService.updateAcesso(AcessoDto,
id);
        return ResponseEntity.noContent().build();
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Object> deleteAcesso(@PathVariable Long id){
        this.AcessoService.deleteAcesso(id);
        return ResponseEntity.noContent().build();
    }}
```

CaixaController

```
@RestController
@RequestMapping("/caixa")
public class CaixaController {
    @Autowired
    private CaixaService CaixaService;

    @GetMapping
    public ResponseEntity<List<Caixa>> getAllCaixas(){
        return this.CaixaService.getAllCaixas();
    }
    @GetMapping("/findAllIsClosedFalse")
    public ResponseEntity<List<Caixa>> findAllIsClosedFalse(){
        return this.CaixaService.findAllByIsClosedFalse();
    }
    @PutMapping("/CloseCaixa/{id}")
    public ResponseEntity<Caixa> closeCaixa(@PathVariable Long id){
        return this.CaixaService.closeCaixa(id);
    }}
```

CategoriaProdutoController

```
@RestController
@RequestMapping("/categoriaProduto")
public class CategoriaProdutoController {
    @Autowired
    private CategoriaProdutoService CategoriaProdutoService;
    @PostMapping
    public ResponseEntity<CategoriaProduto>
createCategoriaProduto(@RequestBody @Valid CategoriaProdutoDto
CategoriaProdutoDto){
        CategoriaProduto CategoriaProduto =
this.CategoriaProdutoService.createCategoriaProduto(CategoriaProdutoDto);
        return new ResponseEntity<>(CategoriaProduto,
HttpStatus.CREATED);
    }
    @GetMapping
    public ResponseEntity<List<CategoriaProduto>>
getAllProductCategories(){
        return this.CategoriaProdutoService.getAllProductCategories();
    }
```

```
    @PutMapping("/{id}")
    public ResponseEntity<CategoriaProduto>
updateCategoriaProduto(@RequestBody @Valid CategoriaProdutoDto
CategoriaProdutoDto, @PathVariable Long id) throws Exception {
        CategoriaProduto CategoriaProduto =
this.CategoriaProdutoService.updateCategoriaProduto(CategoriaProdutoDto,
id);
        return ResponseEntity.noContent().build();
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Object> deleteCategoriaProduto(@PathVariable
Long id){
        this.CategoriaProdutoService.deleteCategoriaProduto(id);
        return ResponseEntity.noContent().build();
    }}
```

ClienteController

```
@RestController()
@RequestMapping("/cliente")
public class ClienteController {
    @Autowired
    private ClienteService ClienteService;

    @PostMapping
    public ResponseEntity<Cliente> createCliente(@RequestBody @Valid
ClienteDto ClienteDto) throws Exception{
        Cliente newCliente =
this.ClienteService.createCliente(ClienteDto);
        return new ResponseEntity<>(newCliente, HttpStatus.CREATED);
    }
    @GetMapping
    public ResponseEntity<List<Cliente>> getAllClientes(){
        return this.ClienteService.getAllClientes();
    }
    @PutMapping("/{id}")
    public ResponseEntity<Object> updateCliente(@RequestBody @Valid
ClienteDto ClienteDto, @PathVariable Long id) throws Exception{
        Cliente newCliente =
this.ClienteService.updateCliente(ClienteDto, id);
        return ResponseEntity.noContent().build();
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Object> disableCliente(@PathVariable Long id){
        this.ClienteService.disableCliente(id);
        return ResponseEntity.noContent().build();
    }}
```

FornecedorController

```
@RestController()
@RequestMapping("/fornecedor")
public class FornecedorController {
    @Autowired
    private FornecedorService FornecedorService;

    @PostMapping
    public ResponseEntity<Fornecedor> createFornecedor(@RequestBody
@Valid FornecedorDto FornecedorDto) throws Exception{
        Fornecedor newFornecedor =
this.FornecedorService.createFornecedor(FornecedorDto);
        return new ResponseEntity<>(newFornecedor, HttpStatus.CREATED);
```

```
    }
    @GetMapping
    public ResponseEntity<List<Fornecedor>> getAllFornecedors(){
        return this.FornecedorService.getAllFornecedors();
    }
    @PutMapping("/{id}")
    public ResponseEntity<Object> updateFornecedor(@RequestBody @Valid
FornecedorDto FornecedorDto, @PathVariable Long id) throws Exception{
        Fornecedor newFornecedor =
this.FornecedorService.updateFornecedor(FornecedorDto, id);
        return ResponseEntity.noContent().build();
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Object> disableFornecedor(@PathVariable Long
id){
        this.FornecedorService.disableFornecedor(id);
        return ResponseEntity.noContent().build();
    }}
```

ItemPedidoController

```
@RestController()
@RequestMapping("/itemPedido")
public class ItemPedidoController {
    @Autowired
    private ItemPedidoService ItemPedidoService;

    @PostMapping
    public ResponseEntity<ItemPedido> createSell(@RequestBody @Valid
ItemPedidoDto ItemPedidoDto) throws Exception{
        ItemPedido ItemPedido =
this.ItemPedidoService.createItemPedido(ItemPedidoDto);
        return new ResponseEntity<>(ItemPedido, HttpStatus.OK);
    }
    @GetMapping
    public ResponseEntity<List<ItemPedido>> getAllItemPedidos(){
        return this.ItemPedidoService.getAllItemPedidos();
    }}
```

MetodoPagamentoController

```
@RestController
@RequestMapping("/metodoPagamento")
public class MetodoPagamentoController {
    @Autowired
    private MetodoPagamentoService MetodoPagamentoService;
    @PostMapping
    public ResponseEntity<MetodoPagamento>
createMetodoPagamento(@RequestBody @Valid MetodoPagamentoDto
MetodoPagamentoDto){
        MetodoPagamento newMetodoPagamento =
this.MetodoPagamentoService.createMetodoPagamento(MetodoPagamentoDto);
        return new ResponseEntity<>(newMetodoPagamento,
HttpStatus.CREATED);
    }
    @GetMapping
    public ResponseEntity<List<MetodoPagamento>>
getAllMetodoPagamentos(){
        return this.MetodoPagamentoService.getAllMetodoPagamentos();
    }
    @GetMapping("/{id}")
    public ResponseEntity<MetodoPagamento>
```

```
getMetodoPagamentoById(@PathVariable Long id) throws Exception {
       MetodoPagamento MetodoPagamento =
MetodoPagamentoService.findMetodoPagamentoById(id);
       return new ResponseEntity<>(MetodoPagamento, HttpStatus.OK);
    }

@GetMapping("/descriptionMetodoPagamento/{descriptionMetodoPagamento}")
    public ResponseEntity<MetodoPagamento>
getMetodoPagamentoByDescriptionMetodoPagamento(@PathVariable String
descriptionMetodoPagamento) throws Exception {
        MetodoPagamento MetodoPagamento =
MetodoPagamentoService.findMetodoPagamentoByDescriptionMetodoPagamento(de
scriptionMetodoPagamento);
        return new ResponseEntity<>(MetodoPagamento, HttpStatus.OK);
    }
    @PutMapping("/{id}")
    public ResponseEntity<MetodoPagamento>
updateMetodoPagamento(@RequestBody @Valid MetodoPagamentoDto
MetodoPagamentoDto, @PathVariable Long id) throws Exception {
        MetodoPagamento newMetodoPagamento =
this.MetodoPagamentoService.updateMetodoPagamento(MetodoPagamentoDto,
id);
        return ResponseEntity.noContent().build();
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Object> deleteMetodoPagamento(@PathVariable
Long id){
        this.MetodoPagamentoService.deleteMetodoPagamento(id);
        return ResponseEntity.noContent().build();
    }}
```

PedidoController

```
@RestController()
@RequestMapping("/pedido")
public class PedidoController {
    @Autowired
    private PedidoService PedidoService;

    @PostMapping
    public ResponseEntity<Pedido> createSell(@RequestBody @Valid
PedidoDto PedidoDto) throws Exception{
        Pedido Pedido = this.PedidoService.createPedido(PedidoDto);
        return new ResponseEntity<>(Pedido, HttpStatus.OK);
    }
    @GetMapping
    public ResponseEntity<List<Pedido>> getAllPedidos(){
        return this.PedidoService.getAllPedidos();
    }}
```

ProdutoController

```
@RestController()
@RequestMapping("/produto")
public class ProdutoController {
    @Autowired
    private ProdutoService ProdutoService;

    @PostMapping
    public ResponseEntity<Produto> createProduto(@RequestBody @Valid
ProdutoDto ProdutoDto) throws Exception{
        Produto newProduto =
```

```
this.ProdutoService.createProduto(ProdutoDto);
        return new ResponseEntity<>(newProduto, HttpStatus.CREATED);
    }
    @GetMapping
    public ResponseEntity<List<Produto>> getAllProdutos(){
        return this.ProdutoService.getAllProdutos();
    }
    @GetMapping("/{barcode}")
    public ResponseEntity<Produto> getProdutoByBarCode(@PathVariable Long
barcode) throws Exception {
        Produto Produto =
this.ProdutoService.findProdutoByIBarCode(barcode);
        return new ResponseEntity<>(Produto, HttpStatus.OK);
    }
    @PutMapping("/{id}")
    public ResponseEntity<Object> updateProduto(@RequestBody @Valid
ProdutoDto ProdutoDto, @PathVariable Long id) throws Exception{
        Produto newProduto =
this.ProdutoService.updateProduto(ProdutoDto, id);
        return ResponseEntity.noContent().build();
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Object> disableProduto(@PathVariable Long id){
        this.ProdutoService.disableProduto(id);
        return ResponseEntity.noContent().build();
    }}
```

StatusPedidoController

```
@RestController
@RequestMapping("/status")
public class StatusPedidoController {
    @Autowired
    private StatusPedidoService StatusPedidoService;
    @PostMapping
    public ResponseEntity<StatusPedido> createStatusPedido(@RequestBody
@Valid StatusPedidoDto StatusPedidoDto){
        StatusPedido newStatusPedido =
this.StatusPedidoService.createStatusPedido(StatusPedidoDto);
        return new ResponseEntity<>(newStatusPedido, HttpStatus.CREATED);
    }
    @GetMapping
    public ResponseEntity<List<StatusPedido>> getAllStatusPedido(){
        return this.StatusPedidoService.getAllStatusPedido();
    }
    @GetMapping("/{id}")
    public ResponseEntity<StatusPedido> getStatusPedidoById(@PathVariable
Long id) throws Exception {
        StatusPedido StatusPedido =
StatusPedidoService.findStatusPedidoById(id);
        return new ResponseEntity<>(StatusPedido, HttpStatus.OK);
    }
    @GetMapping("/StatusPedidoDescription/{StatusPedidoDescription}")
    public ResponseEntity<StatusPedido>
getStatusPedidoByStatusPedidoDescription(@PathVariable String
StatusPedidoDescription) throws Exception {
        StatusPedido StatusPedido =
StatusPedidoService.findStatusPedidoByStatusPedidoDescription(StatusPedid
oDescription);
        return new ResponseEntity<>(StatusPedido, HttpStatus.OK);
    }
    @PutMapping("/{id}")
```

```
    public ResponseEntity<StatusPedido> updateStatusPedido(@RequestBody
@Valid StatusPedidoDto StatusPedidoDto, @PathVariable Long id) throws
Exception {
        StatusPedido newStatusPedido =
this.StatusPedidoService.updateStatusPedido(StatusPedidoDto, id);
        return ResponseEntity.noContent().build();
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Object> deleteStatusPedido(@PathVariable Long
id){
        this.StatusPedidoService.deleteStatusPedido(id);
        return ResponseEntity.noContent().build();
    }}
```

TipoPedidoController

```
@RestController
@RequestMapping("/tipoPedido")
public class TipoPedidoController {
    @Autowired
    private TipoPedidoService TipoPedidoService;
    @PostMapping
    public ResponseEntity<TipoPedido> createTipoPedido(@RequestBody
@Valid TipoPedidoDto TipoPedidoDto){
        TipoPedido newTipoPedido =
this.TipoPedidoService.createTipoPedido(TipoPedidoDto);
        return new ResponseEntity<>(newTipoPedido, HttpStatus.CREATED);
    }
    @GetMapping
    public ResponseEntity<List<TipoPedido>> getAllTipoPedidos(){
        return this.TipoPedidoService.getAllTipoPedidos();
    }
    @GetMapping("/{id}")
    public ResponseEntity<TipoPedido> getTipoPedidoById(@PathVariable
Long id) throws Exception {
        TipoPedido TipoPedido = TipoPedidoService.findTipoPedidoById(id);
        return new ResponseEntity<>(TipoPedido, HttpStatus.OK);
    }
    @GetMapping("/TipoPedidoDescription/{TipoPedidoDescription}")
    public ResponseEntity<TipoPedido>
getTipoPedidoByTipoPedidoDescription(@PathVariable String
TipoPedidoDescription) throws Exception {
        TipoPedido TipoPedido =
TipoPedidoService.findTipoPedidoByTipoPedidoDescription(TipoPedidoDescrip
tion);
        return new ResponseEntity<>(TipoPedido, HttpStatus.OK);
    }
    @PutMapping("/{id}")
    public ResponseEntity<TipoPedido> updateTipoPedido(@RequestBody
@Valid TipoPedidoDto TipoPedidoDto, @PathVariable Long id) throws
Exception {
        TipoPedido newTipoPedido =
this.TipoPedidoService.updateTipoPedido(TipoPedidoDto, id);
        return ResponseEntity.noContent().build();
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<Object> deleteTipoPedido(@PathVariable Long
id){
        this.TipoPedidoService.deleteTipoPedido(id);
        return ResponseEntity.noContent().build();
    }}
```

CAPÍTULO 4 - RESULTADOS

Após toda a criação dos diagramas, que foi utilizado como base para a criação da API e do banco de dados, podemos avaliar o funcionando do sistema integrando a API com o front-end.

Para a criação das telas, foi utilizado o Java Swing para realizar o visual e interação das telas. Utilizando o padrão de projeto MVC, fica mais fácil identificar como organizar a estrutura do projeto no front-ed. Onde basta seguir o mesmo padrão, separando a camada Model para os dados, a View para a programação das telas, e o Controlller para adicionar as funcionalidades às telas.

Atualmente está funcionando a tela do Menu Principal, Realizar Venda PDV e todas as telas de consultas (Vendas, Produtos, Fornecedores, Clientes e Estoque). Faltando finalizar as telas de cadastros, caixa e venda tipo delivery.

Tela Inicial

Na tela do Menu, é reunida as principais funções do sistema, como Realizar Venda, Consultas, Cadastros, Caixa e Sair. Como identidade visual, optei pela cor azul.

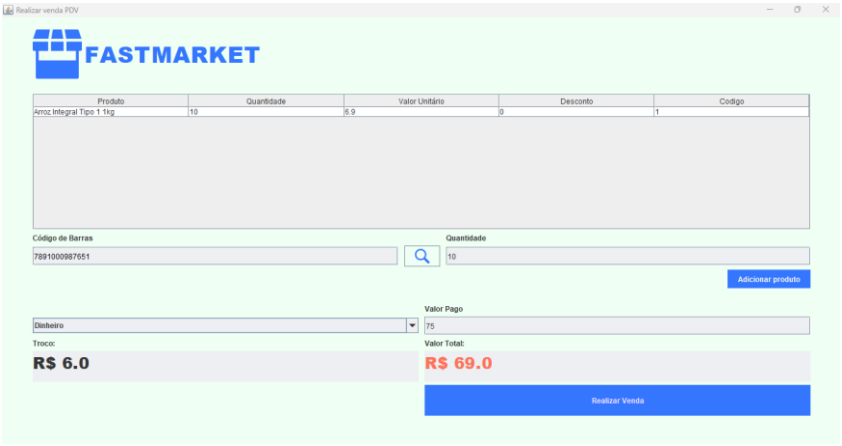


Tela - Venda PDV

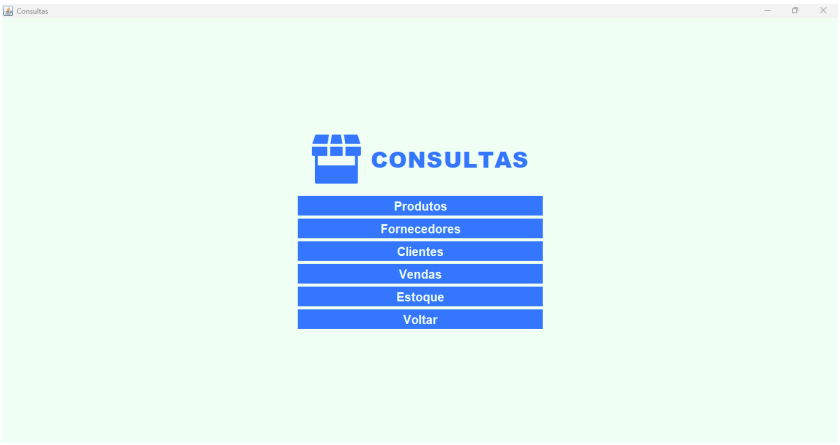
A tela de venda PDV segue a tela tradicional de um sistema de vendas de supermercados, onde o usuário pesquisa o produto através da leitura do código de barras, informa a quantidade e adicionar na venda, onde é listado na tabela principal da tela.

Após isso, é selecionado a forma de pagamento e caso seja troco, irá aparecer um campo para informar o valor pago pelo cliente e gerar o cálculo do troco.

Após isso, basta finalizar a venda.

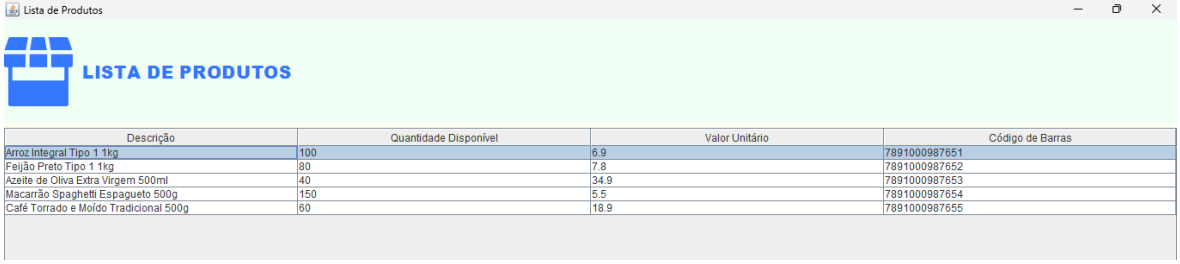


Tela - Menu de Consultas



Tela - Lista de Produtos

Por enquanto, ainda não foi criado as telas de cadastros. Sendo assim, todo cadastrando, com exceção das de vendas, que são realizadas automaticamente ao realizar a venda, são criadas através do MySQL como forma de demonstração das telas de consultas, onde todas já foram criadas.

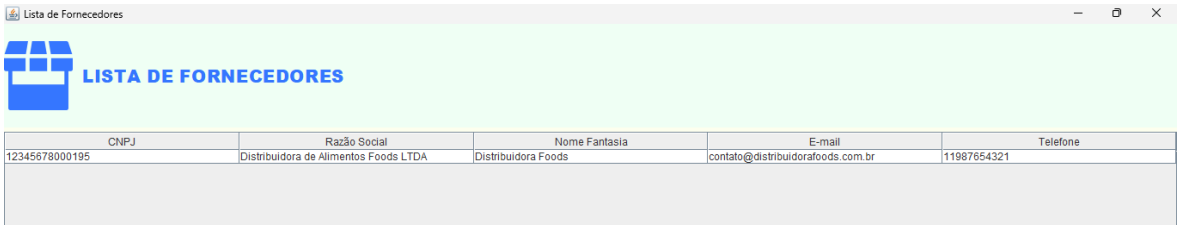


Tela de produtos funcionando, listando todos os produtos cadastrados no banco de dados.

id	codigo_barras	descricao	is_ativo	marca	preco_custo	preco_venda	quantidade	id_categoria_produto	id_fornecedor
1	7891000987651	Arroz Integral Tipo 1 1kg	1	Camil	3.5	6.9	100	1	1
2	7891000987652	Feijão Preto Tipo 1 1kg	1	Kicaldo	4.2	7.8	80	1	1
3	7891000987653	Azeite de Oliva Extra Virgem 500ml	1	Gallo	18	34.9	40	1	1
4	7891000987654	Macarrão Spaghetti Espagueto 500g	1	Barilla	3	5.5	150	1	1
5	7891000987655	Café Torrado e Moído Tradicional 500g	1	Pilão	10.5	18.9	60	1	1
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Tela - Lista de Fornecedores

Nesta tela de Lista de Fornecedores, são apresentados os dados referentes a CNPJ, Razão Social, Nome Fantasia, E-mail e Telefone.

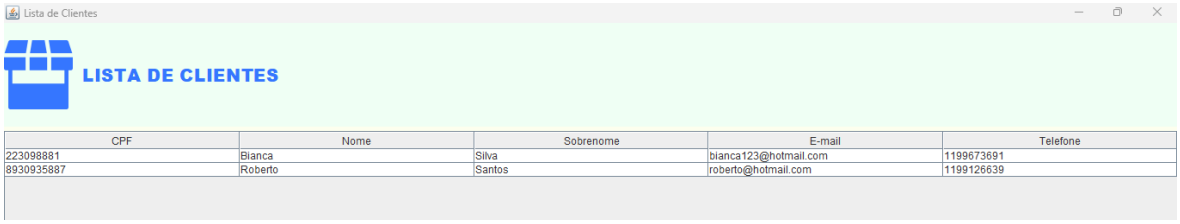


Tela funcionando, listando todos os fornecedores cadastrados no banco de dados.

id	cnpj	email	is_ativo	nome_fantasia	razao_social	telefone
1	12345678000195	contato@distribuidorafoods.com.br	1	Distribuidora Foods	Distribuidora de Alimentos Foods LTDA	11987654321
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Tela - Lista de Clientes

Nesta tela são apresentados os dados referentes a CPF, Nome, Sobrenome, E-mail e Telefone.



Tela funcionando, listando todos os clientes cadastrados no banco de dados.

id	cpf	email	is_ativo	primeiro_nome	telefone	ultimo_nome
1	223098881	bianca123@hotmail.com	1	Bianca	1199673691	Silva
2	8930935887	roberto@hotmail.com	1	Roberto	1199126639	Santos
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Tela – Lista de Vendas

Nesta tela, são apresentados os dados referentes a Data da Venda, Produto, Quantidade, Cliente e Valor. Para as informações ficarem completas, essa tela

mescla dados da tabela TB_PEDIDO, onde contém a informação da data da venda, cliente e tipo de pedido (PDV ou Delivery), com a tabela TB_ITEMPEDIDO, que contém as informações referente ao produto vendido, quantidade, valor final e desconto.

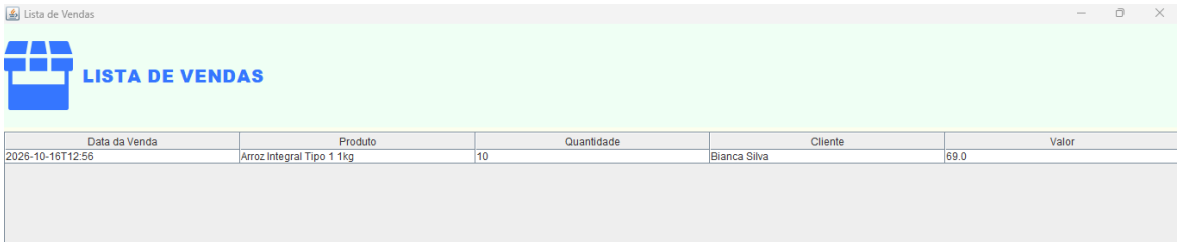


Tabela TB_PEDIDO cadastrada, com as informações de data da venda, cliente e tipo de pedido.

id	data_venda	id_cliente	id_metodo_pagamento	status_pedido_id	tipo_pedido_id
1	2026-10-16 15:56:00.000000	1	1	NULL	1
NULL	NULL	NULL	NULL	NULL	NULL

Conforme o cadastro no banco, o método de pagamento com ID 1, se refere ao Dinheiro, o Tipo de Pedido com ID 1 se refere ao tipo de venda PDV. A coluna status_pedido_id está como null, pois essa coluna é preenchida apenas quando o tipo de venda é Delivery, pois nela são listadas as etapas do pedido.

Tabela TB_ITEM PEDIDO cadastrada, com as informações de desconto, quantidade, valor, id do pedido e id do produto.

id	desconto	quantidade	valor	id_pedido	id_produto
1	0	10	69	1	1
NULL	NULL	NULL	NULL	NULL	NULL

Tela – Lista de Estoque

Nesta tela, são listadas as informações referentes ao produto e sua quantidade, com o objetivo de controlar o estoque.

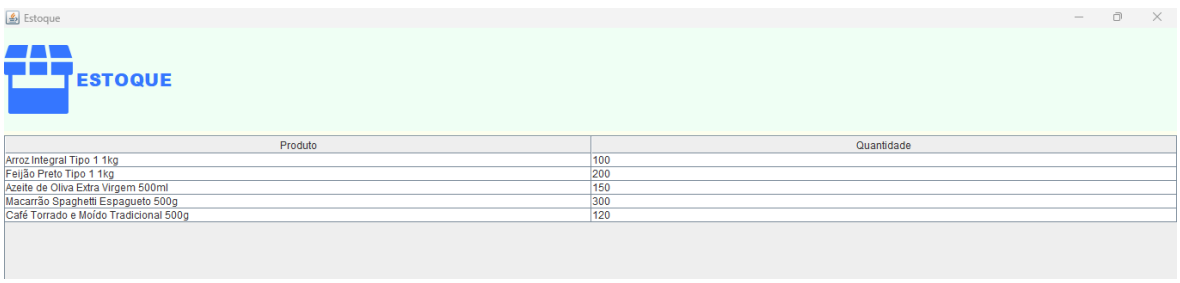


Tabela TB_ESTOQUE cadastrada, com as informações de produto e quantidade.

	id	quantidade	id_produto
▶	1	100	1
	2	200	2
	3	150	3
	4	300	4
	5	120	5
*	NULL	NULL	NULL

Como pendências, falta finalizar as telas de cadastros, venda tipo Delivery e saldo do caixa.

CAPÍTULO 5 – METODOLOGIA

Feedbacks e melhorias

Após a criação e inserção das funcionalidades das telas principais do sistema, introduzi o projeto para um comerciante de Adega poder testá-lo, cadastrei inicialmente os principais produtos que realizam as vendas, como cervejas, energéticos e whisky, instrui como utilizá-lo e acompanhei a utilização dele.

Pude reparar diversas melhorias que poderiam ser inseridas no sistema, incluindo melhorias de navegação de telas, navegação por mouse, onde atualmente o sistema responder melhor as ações do teclado, necessitando de mais ações voltadas ao click do mouse e scroll.

Junto com o usuário, reparei também na necessidade de criar uma segunda tela para acompanhar os pedidos que foram feitas pelo Delivery, pois diferente das vendas PDV, essas podem ser realizadas simultaneamente, sendo necessário a criação de uma tela de consulta dos atuais pedidos em andamento.

Outra melhoria importante, seria na tela de vendas, pois atualmente, o sistema pesquisa e identifica o produto através da leitura do código de barras. Caso não tenha o equipamento que realiza a leitura, é possível abrir a tela de produtos clicando na lupa, e copiar o código de barras do produto, depois voltar na tela de venda e colá-lo. Mas isso não é muito prático no dia a dia, o ideal seria poder além da leitura do código de barras, o sistema permitir pesquisar o produto através do nome ou algum código curto, como costuma ser usado em mercados mais tradicionais.

Testes e validações

Conforme o acompanhamento da utilização do sistema, reparei que o sistema demanda de inserção de diversas validações, como validação de tipo de dado, ou seja, não permitir inserção de letras em campos numéricos. Validação de não permitir a venda de um produto que não está em estoque e se todos os campos obrigatórios estão preenchidos.

Comparação com demais sistemas no mercado

Comparando o sistema do projeto com demais sistemas atuantes no mercado, o do nosso projeto pode ficar atrás em questão de tecnologias mais atuais. Pois utilizamos o Java para a criação tanto da api quanto das telas, onde no back-end, a linguagem atende muito bem, porém em questões de front-end, ela fica um pouco atrás com tecnologias mais recentes e práticas, como Javascript. Isso limita um pouco o desenvolvimento de projetos mais responsivos, onde é possível realizar a programação de forma mais otimizada.

O projeto atual foi pensando em ser utilizado no computador através de uma instalação local. Comparando com demais sistemas, para ficar mais competitivo e atrativo para o mercado, seria interessante uma opção de utilização através de um site, sem necessidade de instalar na máquina. Isso facilitaria futuras atualizações e poderia ser utilizado em outros equipamentos, como tablets e celulares.

Sistemas atuais no mercado também costumam ter relatórios mais robustos, com período de data, visão através de gráficos, como Power BI. Atualmente, o projeto utiliza consultas comuns de vendas já realizadas, clientes cadastrados, produtos cadastrados, entre outros. Seria interessante adicionar mais variedades de filtros e comparativos de vendas utilizando outro período como comparação, para o usuário poder concluir se o período atual vendeu mais ou menos que o período anterior.

REFERÊNCIAS BIBLIOGRÁFICAS

A evolução do PDV! Disponível em: <https://www.linx.com.br/blog/a-evolucaodo-pdv/>. Acesso em: 8 maio. 2026.

CGDW-CRIAÇÃO DE SITE E MARKETING DIGITAL. **RPINFO Sistemas de gestão para Supermercados**. Disponível em: <https://share.google/GkYP4wSs7FEJXwmkr>. Acesso em: 8 maio. 2026.

Covid-19. Disponível em: <https://www.gov.br/saude/pt-br/assuntos/saude-dea-a-z/c/covid-19>. Acesso em: 8 maio. 2026.

DA SILVA, Luiz Filipe Soares. **Ipea - Instituto de Pesquisa Econômica Aplicada**. Disponível em: <https://www.ipea.gov.br/portal/categorias/45-todasas-noticias/noticias/13845-estudo-evidencia-o-impacto-devastador-dapandemia-para-micro-e-pequenas-empresas>. Acesso em: 8 maio. 2026.

DE SÃO PAULO, Acsp –. Associação Comercial. **iFood: como a companhia revolucionou o delivery na pandemia**. Disponível em: <https://acsp.com.br/publicacao/s/ifood-como-a-companhia-revolucionou-odelivery-na-pandemia>. Acesso em: 8 maio. 2026.

DOS SANTOS, Sebastião; ALVES, Felipe Ferreira; DOS SANTOS FILHO, Gilvan Epifânio. RACE - Revista da Administração. **RACE - Revista de Administração do Cesmac**, v. 1, n. 1, 2016.

Entenda as cores para enfrentarmos a Covid 19. Disponível em: <https://www.associacaosabesp.com.br/report/entenda-cores-covid-19>. Acesso em: 8 maio. 2026.

Entregas de comida disparam em 2020 e apps comemoram resultados. Disponível em: <https://g1.globo.com/economia/noticia/2021/02/11/entregasde-comida-disparam-em-2020-e-apps-comemoram-lucros.ghtml>. Acesso em: 8 maio. 2026.

LIFEAPPS. **Diferenças entre e-commerce tradicional e delivery**. Disponível em: <https://share.google/EgfcFBHWhtx81c1ii>. Acesso em: 8 maio. 2026.

LOPES, Nauane Gabriela da Silva. **Estudo de viabilidade econômica de uma fornecedora de marmitas fitness de sistema delivery em Goiânia - GO**. Disponível em: <https://repositorio.pucgoias.edu.br/jspui/handle/123456789/2909>. Acesso em: 8 maio. 2026.

MUNIZ, Mariana. **Plataforma de delivery tem crescimento de 418% em relação a 2020**. Disponível em: <https://veja.abril.com.br/coluna/radar/plataforma-de-delivery-tem-crescimentode-418-em-relacao-a-2020/>. Acesso em: 8 maio. 2026.

NEIVA, Anna. **E-commerce no Brasil 2026: dados e cenário atual - edrone**. Disponível em: <https://edrone.me/br/blog/dados-ecommercebrasil?utm_source=chatgpt.com>. Acesso em: 8 maio. 2026.

O que é Java Spring Boot? Ibm.com, 26 nov. 2025. Disponível em: <https://www.ibm.com/br-pt/think/topics/java-spring-boot>. Acesso em: 8 maio. 2026

Pandemia da Covid-19 se agrava e oito estados adotam toques de recolher. Disponível em: <https://www.cnnbrasil.com.br/saude/pandemia-dacovid-19-se-agrava-e-oito-estados-adotam-toques-de-recolher/>. Acesso em: 8 maio. 2026.

PAPI, Victor. **PDV: quais os tipos e como escolher um software PDV ideal?** Disponível em: <https://transfeera.com/blog/pdv-o-que-e/>. Acesso em: 8 maio. 2026.

PARCEIROS, Ifood Para. **Taxas do iFood: entenda planos, comissão e taxa de serviço para restaurantes**. Disponível em: <https://blogparceiros.ifood.com.br/taxas-ifood/>. Acesso em: 8 maio. 2026.

Prefeito de SP decreta fechamento do comércio a partir de sexta-feira. Disponível em: <https://agenciabrasil.ebc.com.br/geral/noticia/2020-03/prefeitode-

sp-decreta-fechamento-do-comercio-partir-de-sexta-feira>. Acesso em: 8 maio. 2026.

PUBLISHNEWS. **Segunda onda da pandemia fecha o comércio em SP, MG, DF e GO**. Disponível em: <https://www.publishnews.com.br/materias/2021/03/04/segunda-onda-dapandemia-fecha-o-comercio-em-sp-mg-df-e-go>. Acesso em: 8 maio. 2026.

RACE - Revista da Administração. Disponível em: <https://revistas.cesmac.edu.br/administracao/article/view/523/428>. Acesso em: 8 maio. 2026.

SANDER, Carlos. **Metodologias de pesquisa: tipos e exemplos**. **Frons**, 23 abr. 2025. Disponível em: <https://frons.com.br/blog/processos/metodologiasde-pesquisa/>. Acesso em: 8 maio. 2026

Disponível em: <https://economia.uol.com.br/empreendedorismo/noticias/redacao/2025/06/25/ delivery-proprio-ou-aplicativo-o-que-levar-em-conta-na-hora-da-escolha.html>. Acesso em: 8 maio. 2026a.

Disponível em: <https://periodicos.uniarp.edu.br/index.php/visao/article/view/3366>. Acesso em: 8 maio. 2026b.

Disponível em: <https://repositorio.usp.br/item/003026853>. Acesso em: 8 maio. 2026c.

Disponível em: <https://sweda.com.br/blog/solucoes/caixa-registradoraevolucao/#:~:text=A%20evolu%C3%A7%C3%A3o%20da%20caixa%20registra dora,bobina%20da%20caixa%20registrava%20tudo>. Acesso em: 8 maio.
2026d.

Disponível em: <https://repositorio.ipea.gov.br/entities/publication/fc8abae175a8-4c36-9273-26a77e25d5ed>. Acesso em: 8 maio. 2026e.

Disponível em: <http://biblio.direito.ufmg.br/quais-sao-os-instrumentos-decoleta-de-dados-de-pesquisa/>. Acesso em: 8 maio. 2026f.

Disponível em: <https://scholar.archive.org/work/37oijhewezgqffovrs5g6iu5t4/access/wayback/https://www.brazilianjournals.com/index.php/BRJD/article/download/30081/23684>. Acesso em: 9 maio. 2026.

Disponível em: <https://www.pucsp.br/sites/default/files/download/posgraduacao/programas/fonoaudiologia/normas_abnt.pdf>. Acesso em: 9 maio. 2026.

Ciclo de Vida do Desenvolvimento de Software. Disponível em: <https://www.microsoft.com/pt-br/power-platform/topics/phases-of-the-softwaredevelopment-lifecycle>. Acesso em: 9 maio. 2026.

CHAGAS, Igor**. Design Patterns: o guia definitivo dos Padrões de Projeto.** Alura | Cursos Online de Tecnologia, , 21 out. 2021. Disponível em: <https://www.alura.com.br/artigos/design-patterns-introducao-padroesprojeto?srsltid=AfmBOopJcp4hq1zzVBpumkA6nj0TnYrsJzDd8WrKCn2YPuGH bsRsGJb->. Acesso em: 9 maio. 2026

About draw.io. Disponível em: <https://www.drawio.com/docs/about/>. Acesso em: 8 set. 2026.

Spring data JPA. Disponível em: <https://spring.io/projects/spring-data-jpa>. Acesso em: 8 set. 2026.

DIAS, Erickson A. C. **Clean Architecture: Compreendendo e Aplicando a Arquitetura Limpa**. Disponível em: <https://medium.com/@erickson_dias/clean-architecture-compreendendo-e-aplicando-a-arquitetura-limpa-9ff1e84678cd>. Acesso em: 25 set. 2026.

Disponível em: <https://lucid.co/pt/diagrama/uml/tutorial>. Acesso em: 8 set. 2026.

GUIMARÃES, Willyan. **Usando Records em java**. Disponível em: <https://medium.com/experiencecode/usando-records-em-java-9afecf7495b3>. Acesso em: 25 set. 2026.

CRONOGRAMA

1. Escolha e delimitação do tema, linha de pesquisa e contextualização: **Data de entrega: 13/03/26**
2. Tópico Introdução: **Data de entrega: 27/03/26**
3. Referencial Teórico e Metodologia: **Data de entrega: 17/04/26**
4. Pré-projeto completo incluindo elementos pré-textuais e póstextuais: **Data de entrega: 09/05/26**
5. Análise e Levantamento de requisitos e arquitetura do sistema: **Data de entrega: 15/08/26**
6. Diagramas e fluxogramas: **Data de entrega: 25/08/26**
7. Criação do banco de dados: **Data de entrega: 15/09/26**
8. Criação da API: **Data de entrega: 30/09/26**
9. Criação das telas: **Data de entrega: 20/09/26**
10. Programação das telas: **Data de entrega: 15/10/26**
11. Testes e correções: **Data de entrega: 30/10/26**
12. Documentação do código: **Data de entrega: 20/11/26**

ANEXOS

ANEXO A - Desafios na inserção da indústria 4.0 em Micro e Pequenas Empresas



Fonte: Brazilian Journal of Development, Curitiba, v.7, n.5, p. 50614-50637 may 2021